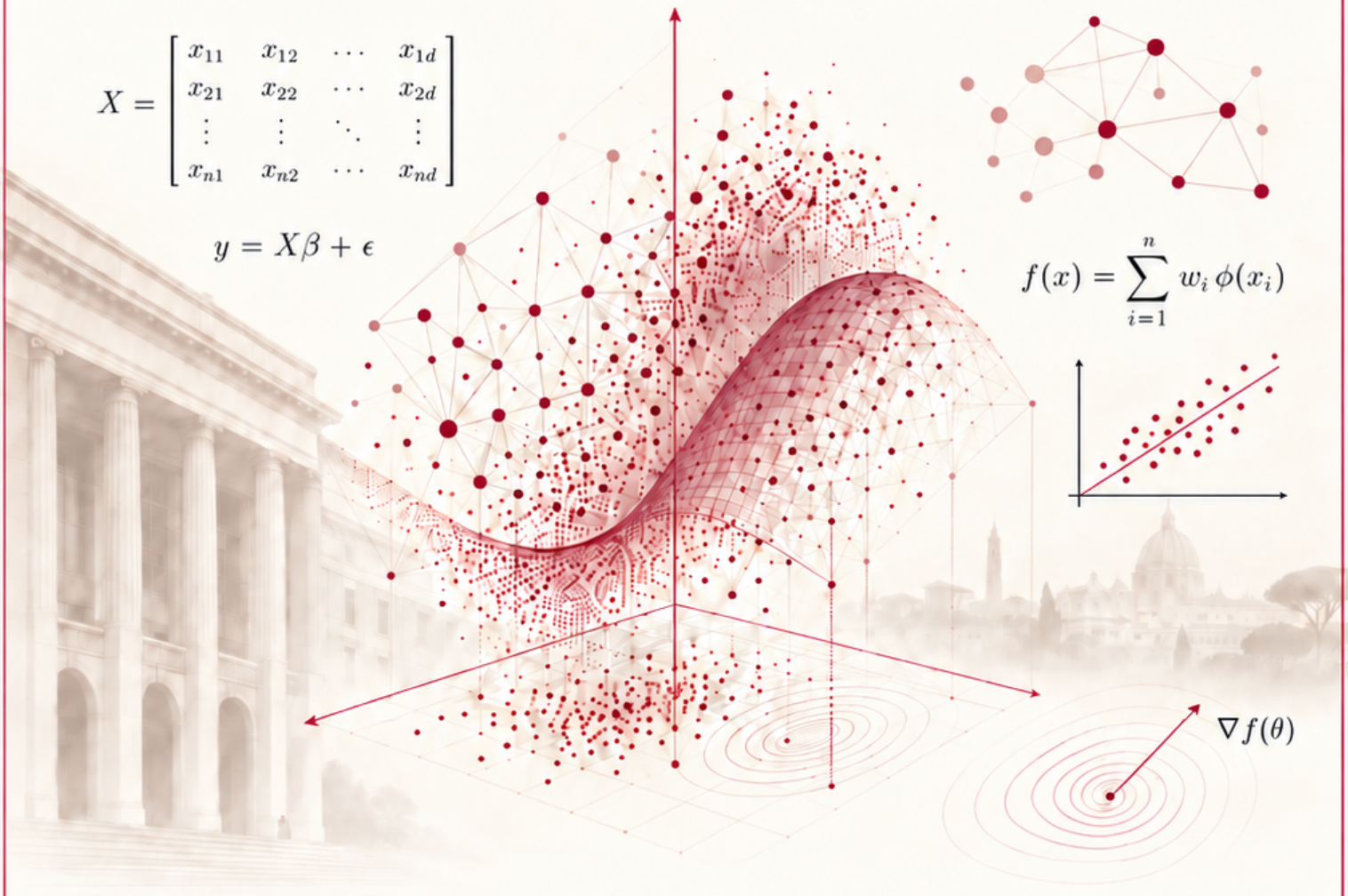
FOUNDATIONS OF DATA SCIENCE

Comprehensive Study Notes



Department of Computer Science

NOTES PREPARED BY

Matt Clifford Trinidad Ramos

Contents

Reading Guide and Notation iv

I Data Science and Machine Learning Foundations 1

1 What Is Data Science? 1

2 Machine Learning as a Subfield of AI 2

2.1 Tom Mitchell's learning-task formulation . . . 2

2.2 Why machine learning? . . . 2

2.3 Machine learning and statistics . . . 3

3 Learning Paradigms 3

4 Supervised Learning 3

5 Representative Supervised-Learning Tasks 3

5.1 Regression . . . 4

5.2 Classification . . . 4

5.3 A supervised model as a family of equations . . . 6

5.4 Structured and high-dimensional outputs . . . 6

5.5 Sequence and cross-modal generation . . . 7

6 Unsupervised Learning and Generative Models 8

6.1 Latent variables . . . 9

7 Reinforcement Learning 10

7.1 Chess as an example . . . 10

7.2 Why reinforcement learning is difficult . . . 10

8 Deep-Learning Landmarks 11

II Data Representation and Linear Algebra Foundations 12

9 The Iris Dataset and Supervised-Learning Data 12

9.1 Features and target . . . 12

9.2 Data types and numerical encoding . . . 12

9.3 Labels and dataset notation . . . 13

10 Data Generation and the i.i.d. Assumption 13

11 Data Points as Vectors 13

12 Spanning Sets and Bases 14

12.1 Spanning sets . . . 14

12.2 Basis . . . 14

13 Standard and Orthonormal Bases 15

13.1 The standard basis . . . 15

13.2 Encoding with an orthonormal basis . . . 16

14 Non-Standard Bases and Coordinate Transformations 16

15 Finding the Encoding Weights 17

16 Subspaces and Projection 18

16.1 Encoding, decoding, and projection . . . 19

III Optimization and Automatic Differentiation 21

17 Optimization Fundamentals 21

17.1 Global and local optima . . . 21

17.2 Zero-order and first-order conditions . . . 21

18 Local Linear Approximation and the Gradient **22**
18.1 Directions of steepest ascent and descent . . . 23
19 Computing Derivatives **24**
19.1 Computation graphs . . . 24
19.2 Forward-mode automatic differentiation . . . 25
19.3 Reverse mode and backpropagation . . . 25
20 Gradient Descent **26**
20.1 Convex and nonconvex objectives . . . 27
20.2 Choosing the step size . . . 28
20.3 Cost-function history plots . . . 29
20.4 Stopping criteria . . . 30
20.5 Weaknesses of vanilla gradient descent . . . 30
20.6 Constrained gradient updates for PCA . . . 31

IV Principal Component Analysis and Dimensionality Reduction **32**
21 Linear Transformations and Projection **32**
22 From a Fixed Basis to a Learned Basis **32**
22.1 Reconstruction error and variance . . . 33
23 Covariance and the First Principal Component **33**
24 Eigenvalues, Eigenvectors, and Principal Components **34**
25 Dimensionality Reduction and Unsupervised Representation Learning **35**
25.1 A compact PCA exercise . . . 36
25.2 Worked solution: from raw samples to reconstruction . . . 36
26 Linear Structure, Manifolds, and Charts **37**

V Linear Models, Generalization, and Evaluation **39**
27 Function Approximation and Risk Minimization **39**
28 Ordinary Least-Squares Regression **39**
28.1 Model, residuals, and squared loss . . . 39
28.2 Derivatives and gradient updates . . . 40
28.3 Normal equations and the geometry of the solution . . . 40
28.4 Regression evaluation metrics . . . 41
29 Feature Maps and Model Capacity **42**
29.1 Linear parameters, nonlinear input relationships . . . 42
29.2 Underfitting and overfitting . . . 42
30 Maximum Likelihood and Probabilistic Prediction **43**
30.1 Likelihood and negative log-likelihood . . . 43
30.2 Gaussian noise recovers least squares . . . 43
31 Binary Logistic Regression **43**
31.1 Scores, probabilities, and the decision boundary . . . 43
31.2 Bernoulli likelihood and binary cross-entropy . . . 44
31.3 Gradient and optimization . . . 44
32 Multiclass Logistic Regression and Softmax **45**
33 Bias, Variance, and Expected Prediction Error **45**
33.1 Repeated training sets . . . 45
33.2 Deriving the squared-error decomposition . . . 45

34 Regularization: Ridge and Lasso **46**
34.1 Penalizing complexity . . . 46
34.2 Ridge regression and a Gaussian prior . . . 46
34.3 Lasso and sparsity . . . 47
35 Model Selection and Honest Evaluation **47**
35.1 Training, validation, and test sets . . . 47
35.2 k-fold cross-validation . . . 48
36 Classification Metrics and Decision Thresholds **48**
36.1 The confusion matrix . . . 48
36.2 Accuracy, precision, recall, specificity, and F1 . . . 49
36.3 ROC curves and area under the curve . . . 49

Quick Review and Terminology **51**

References **52**

Reading Guide and Notation

These notes follow the course from learning paradigms to linear algebra, optimization, PCA, and linear prediction. Each part ends with a short set of takeaways. Use the final review tables for recall, and the worked examples to check calculations.

Definitions and main concepts

Teal terms identify concepts when they are introduced. Teal panels collect definitions, core formulas, and the main ideas to retain.

Examples and worked calculations

Blue panels contain concrete examples or exercises. In a derivation, read the assumptions first, then the algebra, and finally the interpretation of the result.

Assumptions and cautions

Amber panels flag conditions that change whether a statement is valid. Burgundy headings provide navigation; slate notes clarify notation or add context. Every panel also has a written label, so its meaning does not depend on color.

Shared notation

Symbol	Convention throughout the notes
n, d	Number of observations and number of original input features
$\boldsymbol{x}_i \in \mathbb{R}^d$, y_i	Feature column vector and target of observation i
$X \in \mathbb{R}^{n \times d}$	Original data matrix; observation i is row i, feature j is column j
$\widetilde{X} = [\mathbf{1}\ X]$	Regression design matrix with an intercept column; shape $n \times (d+1)$
$C \in \mathbb{R}^{d \times K}$	K orthonormal directions; $C^\top C = I_K$
$\boldsymbol{z}_i = C^\top \boldsymbol{x}_i$	PCA score vector for a centered observation; the score matrix is XC
$\boldsymbol{\beta}$, λ	Fitted coefficients and regularization strength

Reproduced lecture images retain their original notation. In particular, P samples and N dimensions in some figures correspond to n and d here; a column-oriented data matrix in a figure is the transpose of our row-oriented X. A polynomial feature map uses its own feature matrix $\Phi \in \mathbb{R}^{n \times q}$ and coefficient vector $\boldsymbol{\beta} \in \mathbb{R}^q$.

Part I

Data Science and Machine Learning Foundations

1 What Is Data Science?

Definition

Data science is an interdisciplinary field concerned with extracting actionable insight and knowledge from structured and unstructured data, using methods from statistics, mathematics, computer science, and domain expertise.

Prerequisites. Calculus and linear algebra (including derivatives and matrix–vector products), and basic probability and statistics (including Gaussian distributions, mean, and standard deviation).

Data science is broader than model fitting. A typical lifecycle starts from the problem that needs to be solved, passes through data and model development, and continues after deployment because models must be monitored and may need retraining.

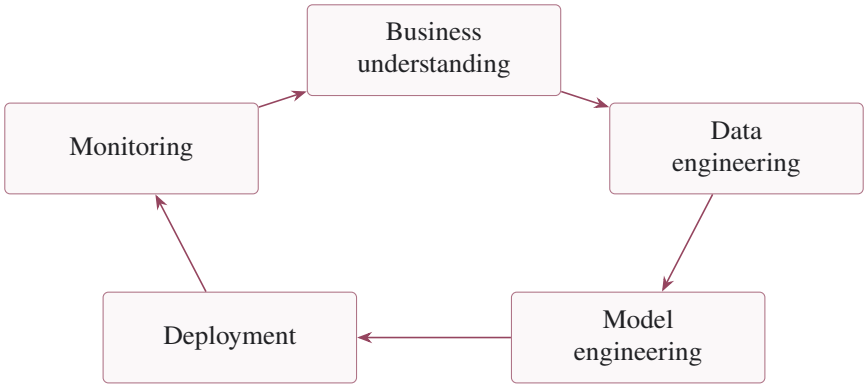


Figure 1. Data science lifecycle from business understanding through data engineering, model engineering, deployment, and monitoring. Feedback from monitoring initiates a new development cycle.

The stages involve the following activities:

- **Business understanding:** define business objectives and translate them into machine-learning objectives.
- **Data engineering:** collect and verify data, clean and visualize it, augment it when appropriate, and standardize it.
- **Model engineering:** select a model, train it, and evaluate it.
- **Deployment:** move the model into production and consider user acceptance and usability.
- **Monitoring:** monitor efficacy, collect new data, and retrain when required.

2 Machine Learning as a Subfield of AI

Artificial Intelligence (AI) is the field of computer science devoted to creating agents capable of tasks that typically require human intelligence, such as learning, reasoning, and problem solving. Supervised learning, unsupervised learning, and reinforcement learning are major machine-learning paradigms within AI. Deep learning cuts across these paradigms.

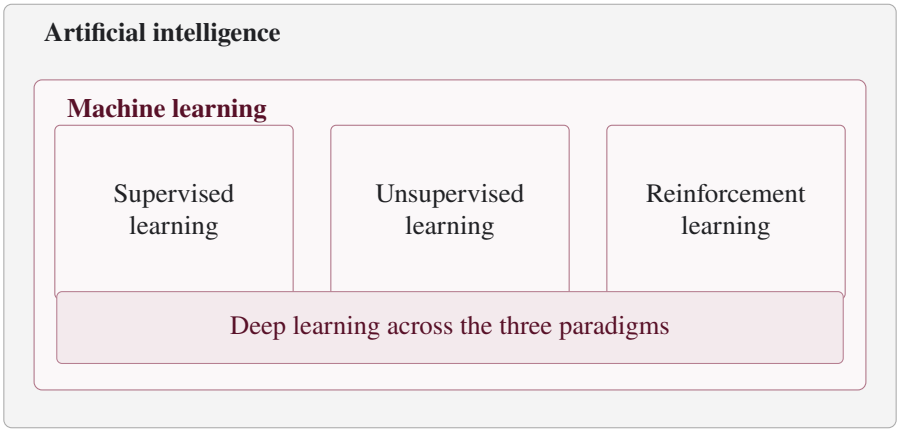


Figure 2. Artificial intelligence, machine learning, and the three learning paradigms. Deep learning supplies model families across the paradigms; the diagram is conceptual rather than a strict set-theoretic taxonomy.

2.1 Tom Mitchell's learning-task formulation

In Tom Mitchell's learning-task formulation, a machine-learning system improves its **performance** P at a **task** T through **experience** E. A well-defined learning problem can therefore be summarized by the triple

$$\langle P, T, E \rangle.$$

Notation. T denotes the task to be performed; P is the quantitative performance measure; E is the experience or data from which the system learns.

Table 1. Examples of tasks, performance measures, and learning experience.

Task T	Performance P	Experience E
Playing checkers	Percentage of games won against an arbitrary opponent	Practice games played against itself
Recognizing handwritten words	Percentage of words correctly classified	Human-labeled images of handwritten words
Driving on four-lane highways using vision sensors	Average distance traveled before a human-judged error	Images and steering commands recorded while observing a human driver
Classifying email as spam or legitimate	Percentage of messages correctly classified	A database of emails with human-provided labels

2.2 Why machine learning?

Many useful tasks are too complex to specify with explicit hand-written rules. Object recognition, speech understanding, and natural-language processing are canonical examples. Instead of manually encoding all rules, machine-learning systems learn useful regularities automatically from data or experience.

2.3 Machine learning and statistics

Statistics and machine learning share substantial mathematical and methodological foundations: both fields uncover patterns in data, rely on calculus, probability, and linear algebra, and share many algorithms. Their differences lie mainly in emphasis: statistics traditionally focuses more strongly on inference, conclusions, interpretability, and rigor, whereas machine learning often emphasizes prediction, scalability, and autonomy. These differences in goals and emphasis do not establish a hard boundary between the disciplines.

3 Learning Paradigms

Table 2. Input/output view of the three paradigms.

Paradigm	Input	Output
Supervised learning	Examples of inputs paired with desired outputs	A model that predicts an output for a new input
Unsupervised learning	Examples of data without target outputs	A representation of structure in the data
Reinforcement learning	A sequence of interactions with an environment	A policy that performs a desired task

4 Supervised Learning

Supervised learning

Supervised learning defines a mapping from input to output and learns this mapping from paired input/output examples. In traditional programming, *data* + *program* produce the output; in machine learning, *data* + *observed outputs* are used to learn the program/model.

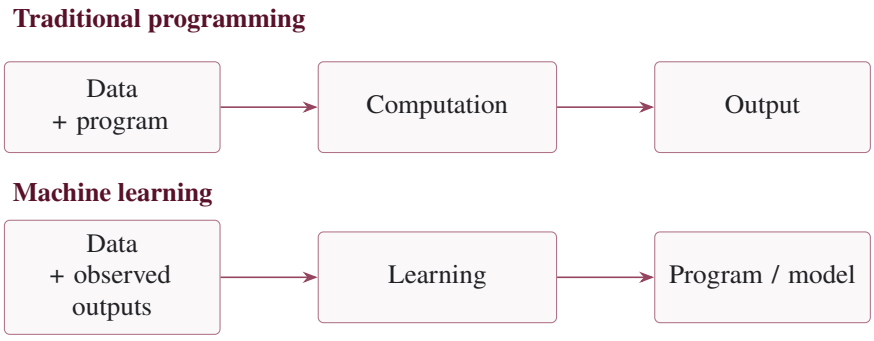


Figure 3. Traditional programming and machine learning: a program computes outputs, whereas learning uses observed inputs and outputs to construct a model.

5 Representative Supervised-Learning Tasks

A common prediction pipeline encodes raw real-world objects numerically, processes them with a model, and decodes the results into meaningful outputs. The examples differ primarily in the structure and dimensionality of the target.

5.1 Regression

Regression predicts real-valued quantities. In the univariate example, real-estate information is encoded as a feature vector and mapped to a single predicted price. In the graph-regression example, a molecular graph is encoded and mapped to more than one continuous target, namely physical properties such as freezing and boiling point.

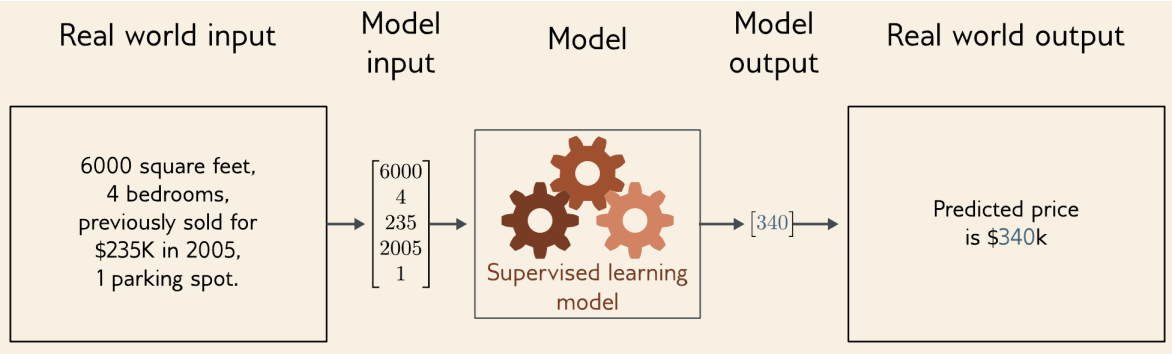


Figure 4. Univariate regression example: a real-world description is encoded numerically and mapped to a single real-valued output.

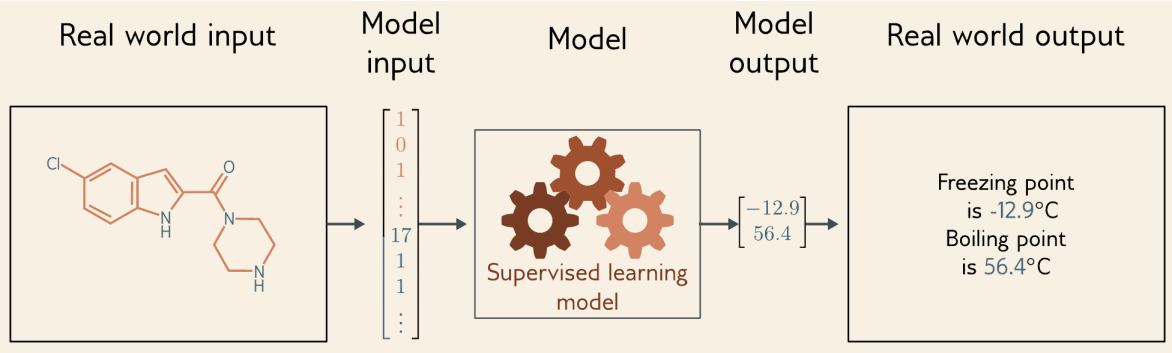


Figure 5. Graph regression example: a molecular structure is represented numerically and mapped to multiple continuous outputs.

5.2 Classification

Classification predicts discrete classes. A binary text-classification example assigns a sentence to one of two classes (positive/negative) using a Transformer. A music-genre example and an image-classification example illustrate multiclass prediction using, respectively, a recurrent neural network (RNN) and a convolutional network.

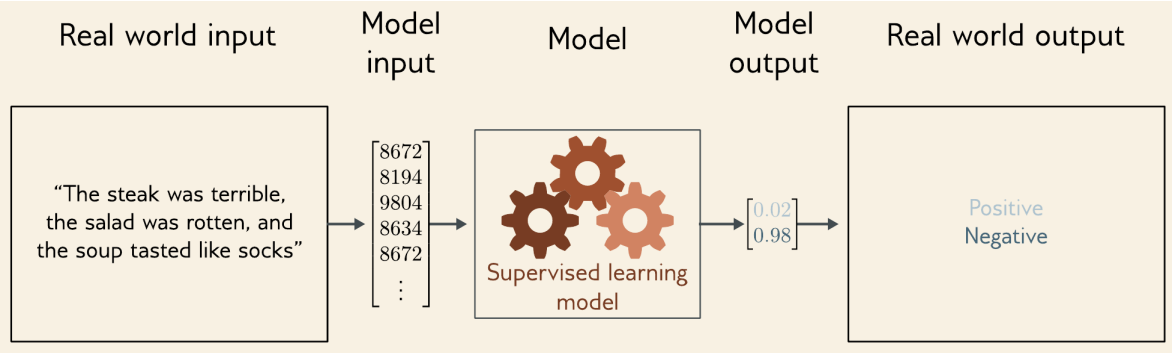


Figure 6. Binary text classification: text → numerical representation → class scores → positive/negative label.

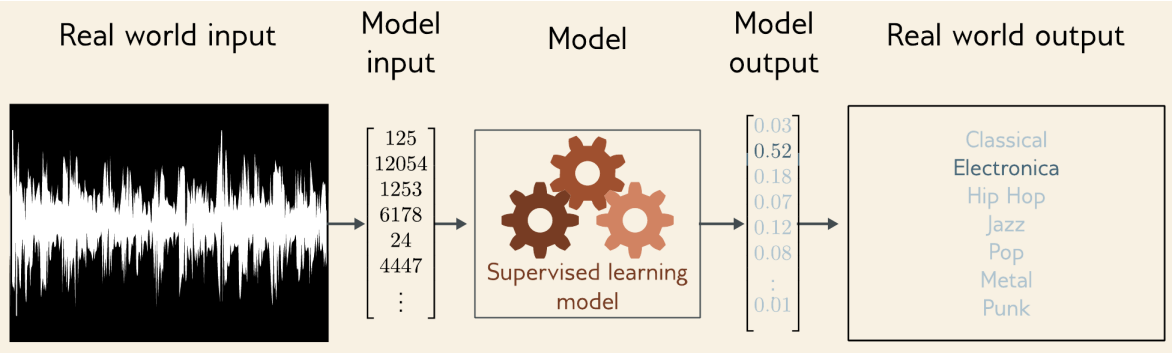


Figure 7. Multiclass music-genre classification using a recurrent neural network.

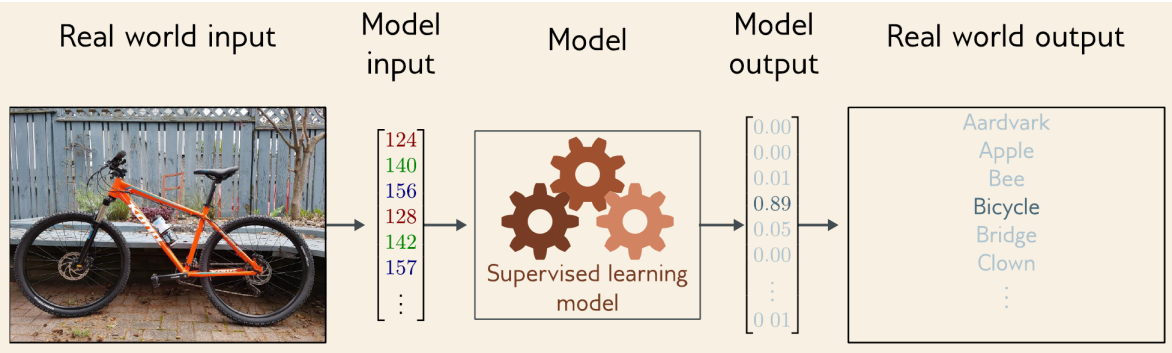


Figure 8. Multiclass image classification using a convolutional network.

5.3 A supervised model as a family of equations

A supervised model can predict height from age. Training means searching through a family of candidate equations and selecting one that fits the training observations well. Deep neural networks form extremely flexible families of equations; fitting such networks is an instance of **deep learning**.

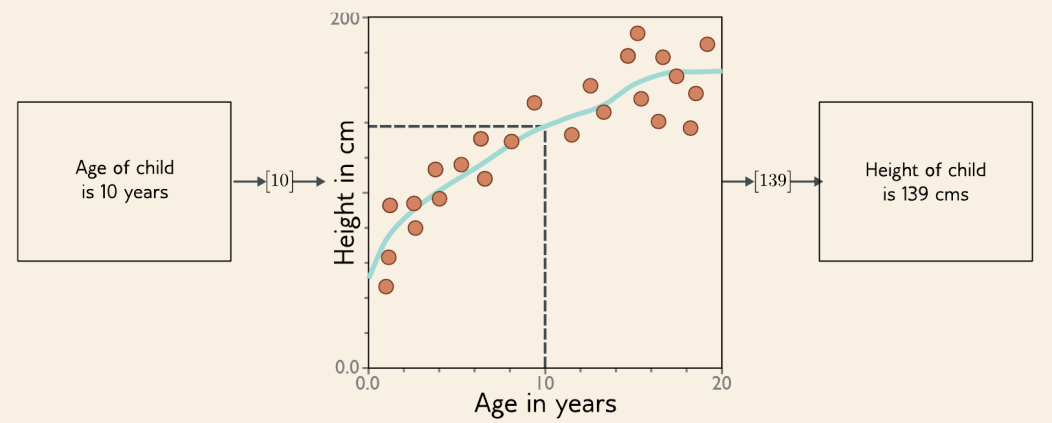


Figure 9. Supervised-learning view of model fitting: an input value is mapped to an output through an equation fitted to training data.

5.4 Structured and high-dimensional outputs

Some supervised tasks produce many outputs rather than a single scalar. Examples include:

- **Image segmentation:** many binary/discrete outputs, one per spatial location; the example uses a convolutional encoder–decoder network.
- **Depth estimation:** many continuous outputs forming a dense depth map; using a convolutional encoder–decoder.
- **Pose estimation:** many continuous outputs describing body/keypoint geometry; using a convolutional encoder–decoder.

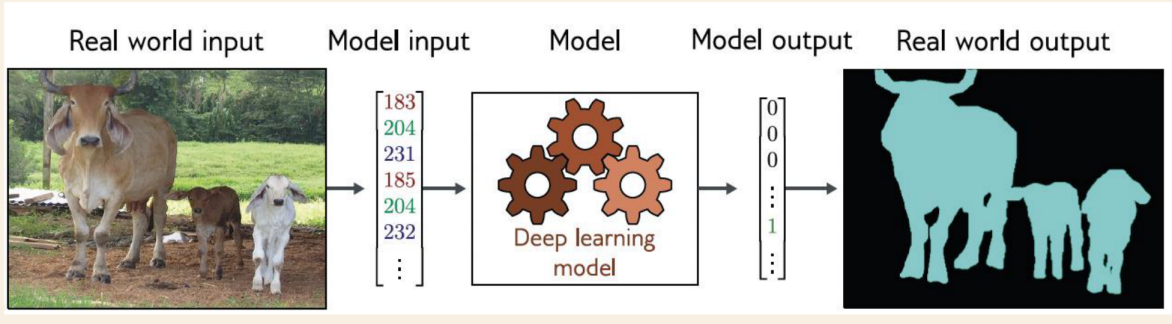


Figure 10. Image segmentation example: an input image is mapped to a dense discrete mask.

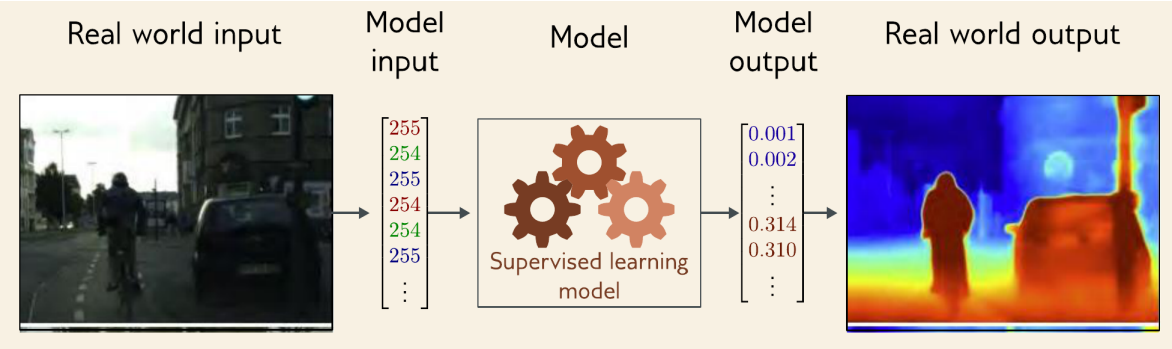


Figure 11. Depth estimation example: an input image is mapped to a dense continuous depth representation.

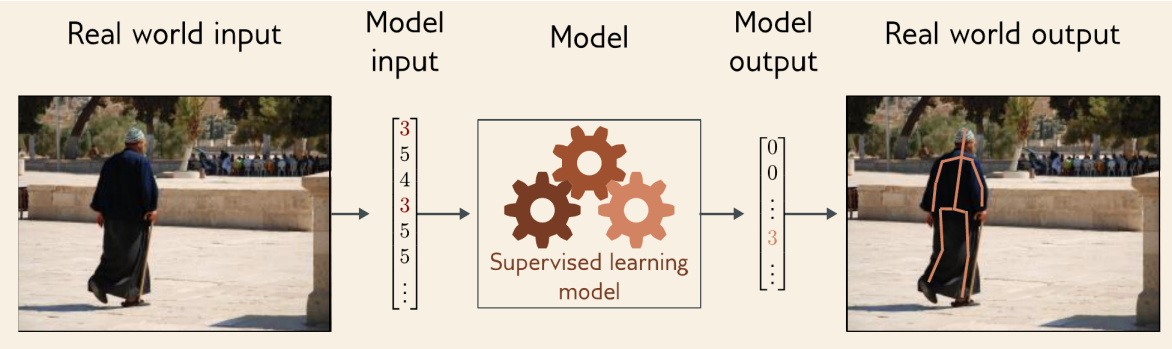


Figure 12. Pose estimation example: an input image is mapped to multiple continuous outputs representing pose.

5.5 Sequence and cross-modal generation

The same input–model–output pattern extends to structured sequences and multimodal tasks. Examples include machine translation (text sequence to text sequence), image captioning (image to text sequence), and text-to-image generation (text sequence to image).

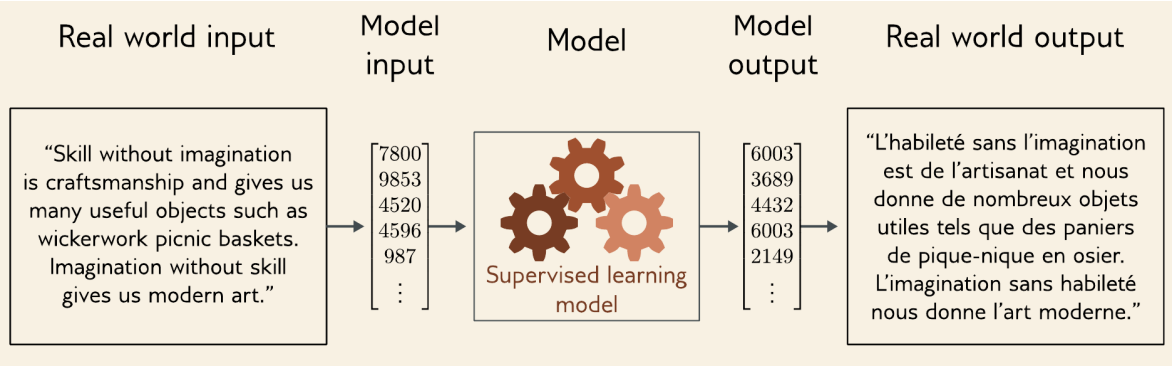


Figure 13. Machine-translation example: text is tokenized to a numerical representation and decoded into a target-language sequence.

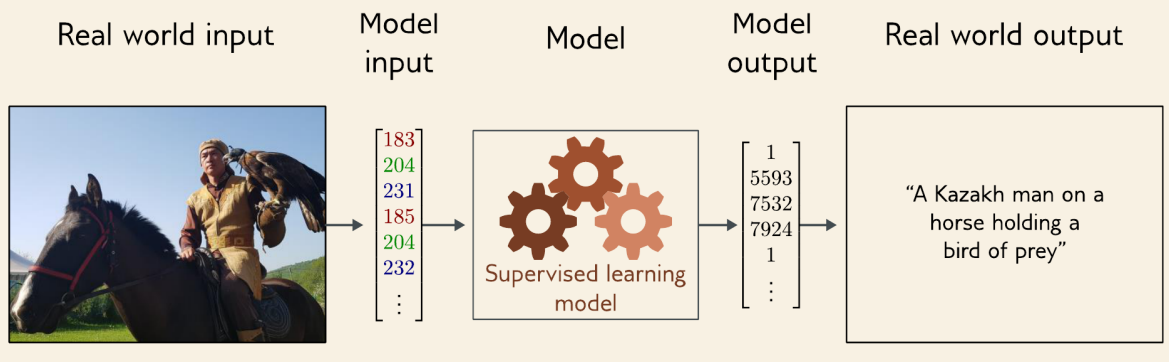


Figure 14. Image-captioning example: an image is encoded numerically and decoded into a textual description.

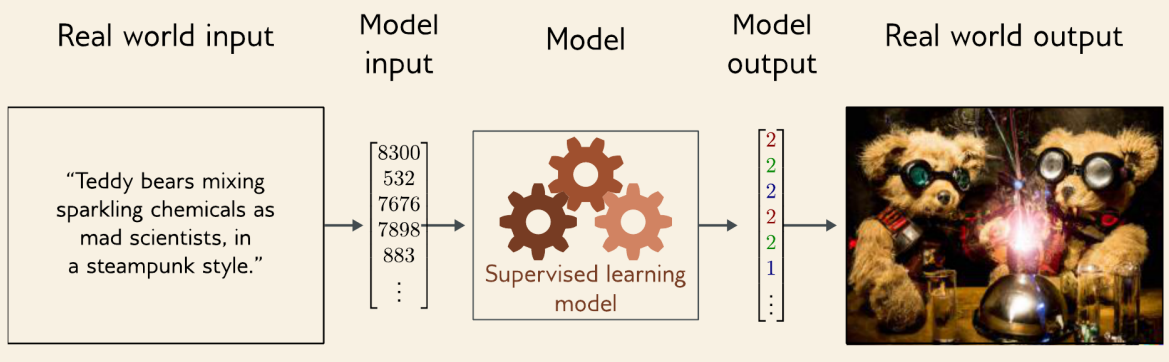


Figure 15. Text-to-image example: a text prompt is encoded and transformed into an image.

Key Concept

Across these examples, the relationship between input and output can be highly complex and there may be multiple valid outputs. Nevertheless, outputs—and often inputs—possess structure: language follows grammatical rules and natural images obey strong statistical regularities. This observation motivates learning the "grammar" of data from large quantities of unlabeled examples.

6 Unsupervised Learning and Generative Models

Unsupervised learning

Unsupervised learning studies datasets without target labels. Representative goals include clustering, outlier detection, generation of new examples, and filling in missing data.

A central idea is to exploit the abundance of unlabeled data to learn structure before or alongside a supervised task. An example is *DeepCluster: Deep Clustering for Unsupervised Learning of Visual Features* (Caron et al., 2018), where visual examples are organized into coherent feature clusters without conventional class labels.

Generative models can create new examples. Approaches include generative adversarial networks (GANs), as well as probabilistic generative models such as variational autoencoders (VAEs), normalizing flows, and diffusion models. Generative modeling is not synonymous with unsupervised learning: conditional generative models may also use labels, paired data, or other forms of supervision.



Figure 16. DeepCluster: visually coherent groups discovered from unlabeled images.

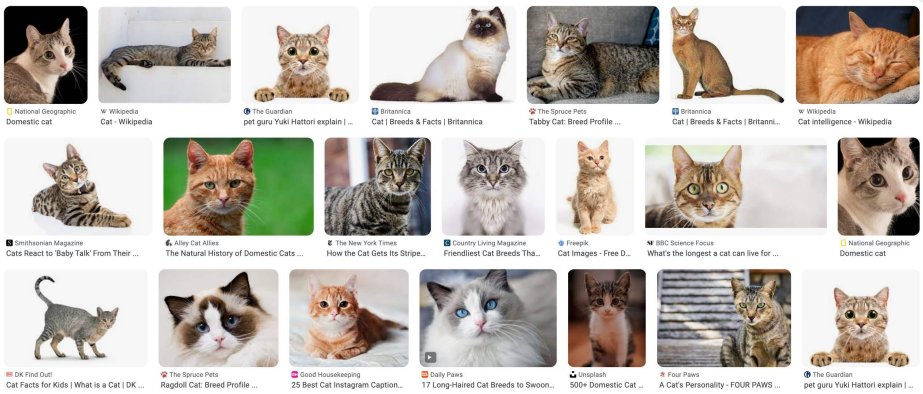


Figure 17. Examples of synthetic cat images produced by generative models.

6.1 Latent variables

A latent-variable generative model represents high-dimensional observed data through lower-dimensional latent variables. Samples are drawn in the latent space, transformed by the model, and decoded into observations.

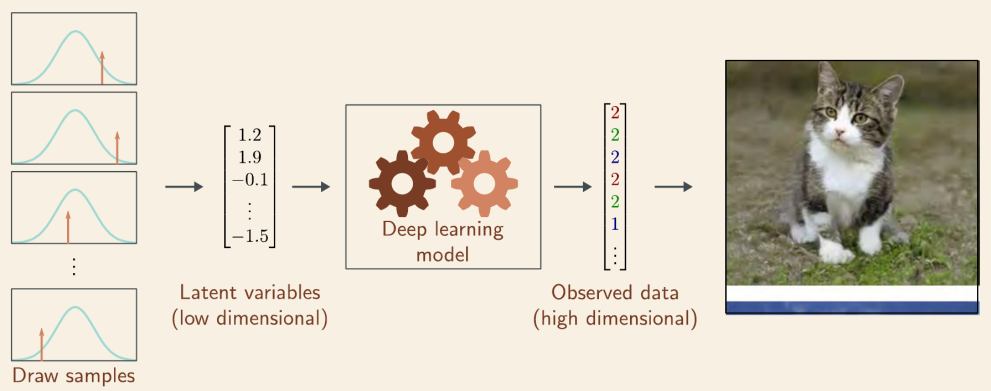


Figure 18. Latent-variable generation: low-dimensional latent samples are transformed by a deep model into high-dimensional observed data.

7 Reinforcement Learning

Reinforcement learning

Reinforcement learning (RL) involves three basic ingredients: a set of **states**, a set of **actions**, and a set of **rewards**. The agent acts in the environment, changes state, and seeks behavior that produces reward. Unlike ordinary supervised learning, the agent is not given a fixed labeled dataset in advance; it must gather information through interaction.

7.1 Chess as an example

In chess, states are valid chess positions and available actions are valid moves. An illustrative reward scheme assigns positive rewards for taking pieces and negative rewards for losing them.

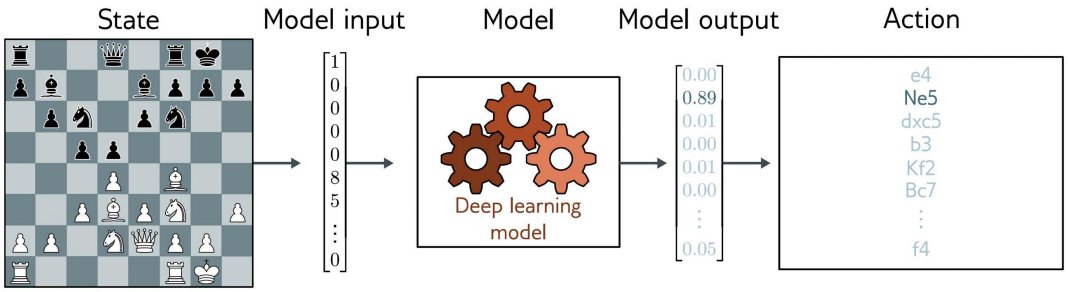


Figure 19. Reinforcement-learning pipeline for chess: board state → model representation → action scores → selected move.

Note

Piece-capture rewards illustrate **reward shaping**: intermediate rewards provide feedback during the state–action–reward interaction loop.

7.2 Why reinforcement learning is difficult

Three central difficulties are:

- **Stochasticity:** the same action need not lead to the same future because other agents or the environment may behave differently.
- **Temporal credit assignment:** a reward observed now may be caused by an earlier sequence of decisions rather than the most recent action alone.
- **Exploration–exploitation trade-off:** an agent must decide whether to reuse a known good strategy or explore alternatives that might be better.

8 Deep-Learning Landmarks

Yoshua Bengio, Geoffrey Hinton, and Yann LeCun received the 2018 ACM A.M. Turing Award. The following timeline summarizes influential developments in neural networks and deep learning.

Table 3. Selected milestones in neural networks and deep learning.

Year	Milestone
1958	Perceptron (simple neural model)
1986	Backpropagation for learning representations in multilayer networks
1989	Convolutional networks (supervised learning)
2012	AlexNet image classification (supervised learning)
2014	Generative adversarial networks (unsupervised learning)
2015	Deep Q-network: human-level control on Atari games
2016	AlphaGo (reinforcement learning)
2017	Transformer architecture for machine translation
2019	GPT-2: language modeling through next-token prediction
2022	DALL-E 2, image synthesis from text prompts
2022	ChatGPT
2023	GPT-4 multimodal model
2024	Demis Hassabis and John Jumper: Nobel Prize in Chemistry in connection with AlphaFold-related work; Geoffrey Hinton and John Hopfield: Nobel Prize in Physics
2026	OpenAI announced an AI-generated proof concerning the Navier–Stokes Millennium problem; formal acceptance remains a separate matter.

Historical note (20 October 2026). OpenAI announced the proof on 8 September 2026; this announcement does not by itself establish a finalized Millennium Prize determination. See OpenAI's announcement and the Clay Mathematics Institute's announcement and evaluation process.

Machine Learning Foundations — Key Takeaways

- Data science covers the full lifecycle from problem definition and data preparation to deployment and monitoring.
- Machine learning replaces explicit hand-coded rules with models learned from data or interaction.
- Supervised, unsupervised, and reinforcement learning differ primarily in the information available during learning and the object being learned.
- Deep learning supplies highly flexible model families that can be used across several learning paradigms.
- Modern tasks often involve structured, high-dimensional inputs and outputs rather than simple scalar prediction.

Part II

Data Representation and Linear Algebra Foundations

9 The Iris Dataset and Supervised-Learning Data

Fisher's Iris dataset provides a compact supervised-learning example. It contains **150 flowers**: 50 *setosa*, 50 *versicolor*, and 50 *virginica*. Each flower is described by four measurements in centimeters: sepal length, sepal width, petal length, and petal width. The target is the iris species.



Figure 20. The three Iris species: versicolor, virginica, and setosa. The classification task predicts the species from sepal and petal measurements.

9.1 Features and target

For n observed objects, the **features** are properties used to describe each object, while the **target** is the output variable or prediction goal. Supervised learning assumes that some relationship exists between features and target and attempts to learn a predictive rule from observed examples.

Table 4. Illustrative Iris data: four measurements in centimeters and the species label.

Sepal L.	Sepal W.	Petal L.	Petal W.	Species
4.3	3.0	1.1	0.1	setosa
5.0	3.3	1.4	0.2	setosa
7.7	3.8	6.7	2.2	virginica
5.5	2.5	4.0	1.3	versicolor

9.2 Data types and numerical encoding

Features and targets can have different types:

- numerical variables in $\mathbb{R}$;
- integer variables in $\mathbb{Z}$;
- categorical variables taking values in a finite set $\{C_1, \ldots, C_g\}$;
- binary variables in $\{0, 1\}$.

For target variables, the distinction between continuous and discrete values leads naturally to regression and classification tasks.

Most learning algorithms expect numerical input. A categorical variable with k mutually exclusive categories can be represented by a one-hot vector

$$\boldsymbol{e}_i = (0, \ldots, 0, \underbrace{1}_{i\text{-th position}}, 0, \ldots, 0)^\top, \qquad i \in \{1, \ldots, k\}. \tag{1}$$

For categories with a natural order, an encoding should reflect that ordinality, for example through an ordered sequence of integers.

Notation. In equation (1), k is the number of categories and $\boldsymbol{e}_i$ is the binary indicator for category i; exactly one coordinate equals 1.

9.3 Labels and dataset notation

Entries in the target column are called **labels**. Data are **labeled** when the target is observed and **unlabeled** when it is unknown.

Let $\mathcal{X} \subseteq \mathbb{R}^d$ be the input space and $\mathcal{Y}$ the target space. Treat the dataset as an indexed sequence, so repeated observations are allowed:

$$Z = ((\boldsymbol{x}_i, y_i))_{i=1}^n \in (\mathcal{X} \times \mathcal{Y})^n, \qquad (\boldsymbol{x}_i, y_i) \in \mathcal{X} \times \mathcal{Y}.$$

Each $\boldsymbol{x}_i = (x_{i1}, \ldots, x_{id})^\top$ is a column vector. The data matrix stores these observations as rows:

$$X = \begin{bmatrix} \boldsymbol{x}_1^\top \\ \vdots \\ \boldsymbol{x}_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}, \qquad X_{:j} = (x_{1j}, \ldots, x_{nj})^\top \in \mathbb{R}^n.$$

Here i indexes observations, j indexes features, and $X_{:j}$ denotes feature column j rather than observation j.

10 Data Generation and the i.i.d. Assumption

The observed dataset Z is assumed to arise from an underlying joint probability distribution P_{XY} defined on $\mathcal{X} \times \mathcal{Y}$. The true distribution is generally unknown. Supervised machine learning seeks to learn enough of its structure to predict y from $\boldsymbol{x}$.

A standard assumption is that samples are drawn independently and identically distributed (i.i.d.) from P_{XY}. This contains two assumptions:

1. **Identically distributed:** every sample follows the same underlying distribution.
2. **Independent:** the realization of one observation does not depend on the other $n - 1$ observations.

This is a strong assumption and may fail in more complex settings such as time series.

11 Data Points as Vectors

For vector representations in the input space $\mathcal{X}$, assume that the dataset has been **mean-centered**: the mean along each input dimension is subtracted so that the data cloud is centered around the origin. This operation is reversible as long as the feature means are retained.

Notation. Throughout these notes, n counts observations and d counts input dimensions. Each $\boldsymbol{x}_i \in \mathbb{R}^d$ is a column vector, while $X \in \mathbb{R}^{n \times d}$ stores observations as rows. Reproduced lecture figures may retain their original symbols: P samples and N dimensions correspond to n and d here.

Each observation can then be viewed as a point—equivalently, a vector—in a multidimensional vector space. A vector space is a collection of vectors closed under vector addition and scalar multiplication.

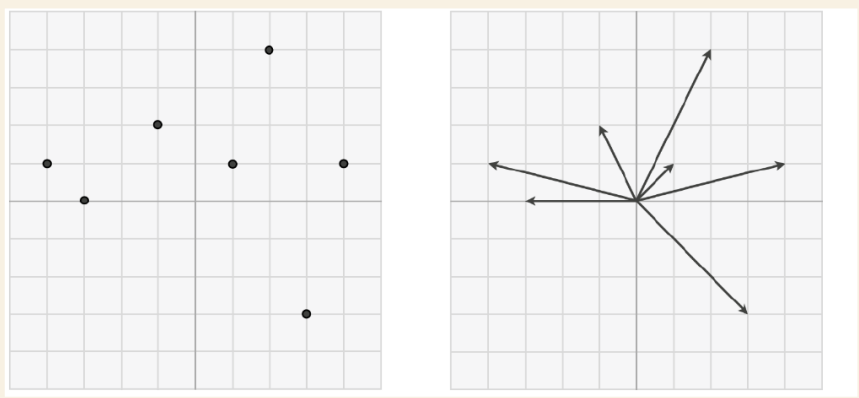


Figure 21. Two equivalent visualizations of the same 2-D data: points (left) and arrows from the origin (right).

12 Spanning Sets and Bases

12.1 Spanning sets

Definition

A **spanning set** is a collection of vectors whose linear combinations can produce every vector in the space being spanned.

In two dimensions, the standard vectors

$$\boldsymbol{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \boldsymbol{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

span the plane because any point $\begin{bmatrix} x & y \end{bmatrix}^\top$ can be written as

$$\begin{bmatrix} x \\ y \end{bmatrix} = x\boldsymbol{e}_1 + y\boldsymbol{e}_2.$$

12.2 Basis

Definition

A **basis** is a linearly independent spanning set. In a d-dimensional vector space, a basis therefore contains exactly d vectors. A more general spanning set may contain more than d vectors, but then it is not a basis in the minimal sense.

For a candidate basis $\boldsymbol{c}_1, \ldots, \boldsymbol{c}_d$ and each data point $\boldsymbol{x}_i$, perfect representation means that there are weights $w_{1,i}, \ldots, w_{d,i}$ such that

$$\sum_{j=1}^{d} \boldsymbol{c}_j w_{j,i} = \boldsymbol{x}_i, \qquad i = 1, \ldots, n.$$

Notation. d is the ambient dimension; n is the number of observations; $\boldsymbol{c}_j$ is basis vector j; $w_{j,i}$ is the coordinate of observation i along $\boldsymbol{c}_j$; and $\boldsymbol{x}_i$ is that observation.

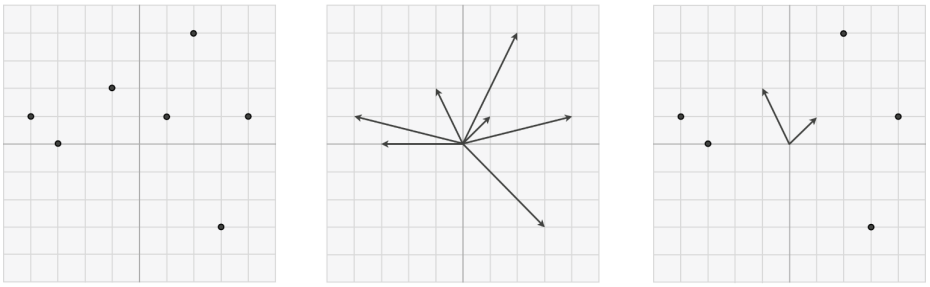


Figure 22. Points, vectors from the origin, and a basis representation in two dimensions.

13 Standard and Orthonormal Bases

13.1 The standard basis

The k-th standard basis vector has zeros everywhere except for a 1 in position k:

$$\boldsymbol{c}_k = (0, \ldots, 0, 1, 0, \ldots, 0)^\top.$$

For distinct basis vectors, the inner product is zero, while each vector has unit Euclidean norm:

$$\begin{aligned} \boldsymbol{c}_j^\top \boldsymbol{c}_m &= 0, & j \neq m, \\ \|\boldsymbol{c}_j\|_2^2 &= 1, & j = 1, \ldots, d. \end{aligned}$$

Thus the standard basis is orthonormal. If

$$C = [\boldsymbol{c}_1 \; \boldsymbol{c}_2 \; \cdots \; \boldsymbol{c}_d],$$

then

$$C^\top C = I_d.$$

In the standard basis, the coordinate vector of a point is simply the point itself: $\boldsymbol{w}_i = \boldsymbol{x}_i$.

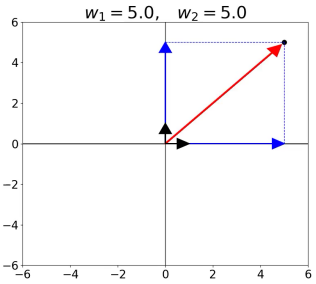


Figure 23. Reconstruction in the two-dimensional standard basis: a point $(5, 5)$ is reconstructed from weights $w_1 = 5$ and $w_2 = 5$.

13.2 Encoding with an orthonormal basis

For an orthonormal basis, the encoding of $\boldsymbol{x}_i$ is obtained directly by inner products:

Orthonormal encoding

$$\boldsymbol{w}_i = C^\top \boldsymbol{x}_i, \qquad i = 1, \ldots, n. \tag{2}$$

Reconstruction is exact when C is a complete $d \times d$ orthonormal basis:

$$CC^\top \boldsymbol{x}_i = \boldsymbol{x}_i, \qquad i = 1, \ldots, n.$$

Notation. C contains the orthonormal basis vectors as columns; $C^\top \boldsymbol{x}_i$ produces the coordinate vector $\boldsymbol{w}_i$; multiplying by C decodes the coordinates back into the original data space.

14 Non-Standard Bases and Coordinate Transformations

For a non-standard basis, the coordinate vector generally differs numerically from the original data point. The encoding is

$$\boldsymbol{w}_i = \begin{bmatrix} w_{1,i} \\ w_{2,i} \\ \vdots \\ w_{d,i} \end{bmatrix},$$

which represents $\boldsymbol{x}_i$ in a transformed feature space whose coordinate axes are defined by the spanning vectors.

Consider the basis vectors

$$\boldsymbol{c}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \boldsymbol{c}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Because these vectors are linearly independent, any point in $\mathbb{R}^2$ can be expressed uniquely as

$$w_{1,i}\boldsymbol{c}_1 + w_{2,i}\boldsymbol{c}_2 = \boldsymbol{x}_i.$$

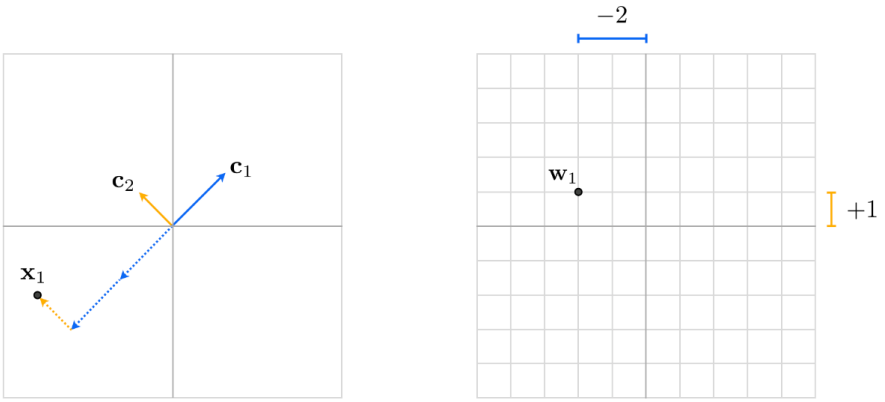


Figure 24. A point $\boldsymbol{x}_1$ in the original coordinates (left) and its encoding $\boldsymbol{w}_1$ in coordinates defined by the non-standard basis (right).

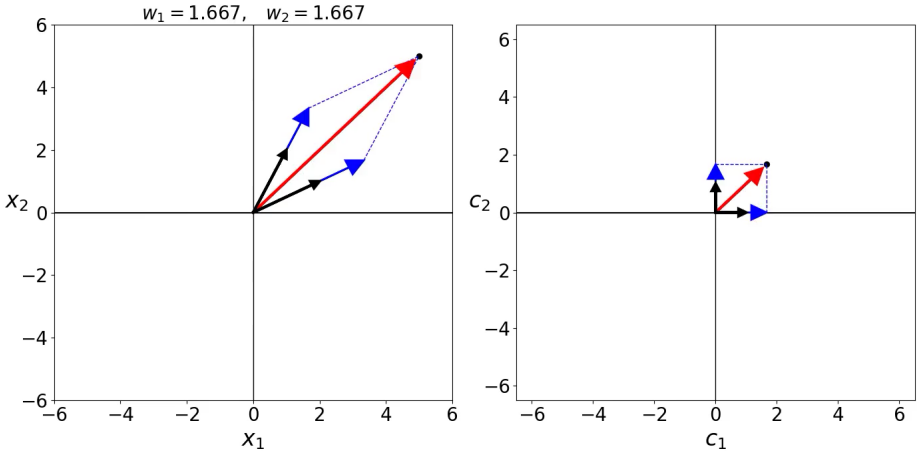


Figure 25. Geometric decomposition in a non-standard basis: the red vector is reconstructed from scaled basis vectors, while the coordinate vector records the required weights.

15 Finding the Encoding Weights

Let the basis/spanning vectors be stacked column-wise into

$$C = [\boldsymbol{c}_1 \; \boldsymbol{c}_2 \; \cdots \; \boldsymbol{c}_d],$$

and let $\boldsymbol{w}_i$ contain the weights for point $\boldsymbol{x}_i$. The squared reconstruction cost is

$$h(\boldsymbol{w}_i) = \|C\boldsymbol{w}_i - \boldsymbol{x}_i\|_2^2.$$

Setting the gradient to zero yields the symmetric first-order system

$$C^\top C\boldsymbol{w}_i = C^\top \boldsymbol{x}_i, \qquad i = 1, \ldots, n. \tag{3}$$

For all n points together, the optimization problem is

$$\underset{\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_n}{\text{minimize}} \; \frac{1}{n} \sum_{i=1}^{n} \|C\boldsymbol{w}_i - \boldsymbol{x}_i\|_2^2.$$

Notation. C is the matrix of spanning vectors; $\boldsymbol{w}_i$ is the coordinate/encoding vector; $\boldsymbol{x}_i$ is the original point; $h(\boldsymbol{w}_i)$ is squared reconstruction error. Equation (3) is the normal-equation form obtained from the first-order optimality condition.

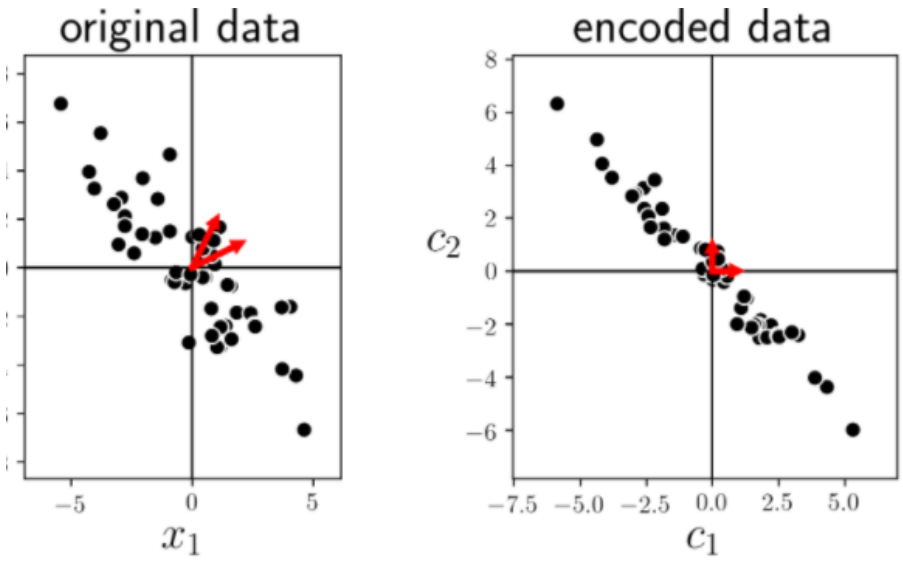


Figure 26. Original data versus the same observations represented in the coordinate system defined by a non-standard spanning set.

16 Subspaces and Projection

Up to this point, perfect representation in a generic d-dimensional space requires enough independent spanning directions. If only $K < d$ independent vectors are used, they can span at most a K-dimensional subspace of the full d-dimensional space. For example, in three dimensions, $K = 2$ independent vectors span a plane, while $K = 1$ spans a line.

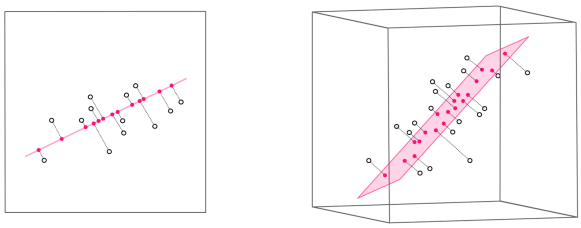


Figure 27. Lower-dimensional subspaces: a line or plane cannot represent every point in the ambient space exactly.

For K spanning vectors, the matrix now has shape

$$C = [\boldsymbol{c}_1 \ \cdots \ \boldsymbol{c}_K] \in \mathbb{R}^{d\times K},$$

but the first-order system keeps the same form:

$$C^\top C\boldsymbol{w}_i = C^\top \boldsymbol{x}_i, \qquad i = 1, \ldots, n.$$

16.1 Encoding, decoding, and projection

When $K < d$, the point cannot in general be reconstructed exactly. Instead, encoding maps the point to K coordinates, and decoding combines those coordinates with the K basis vectors to obtain the closest representable point in the subspace. Geometrically, this is an orthogonal **projection**: dropping the point perpendicularly onto the subspace.

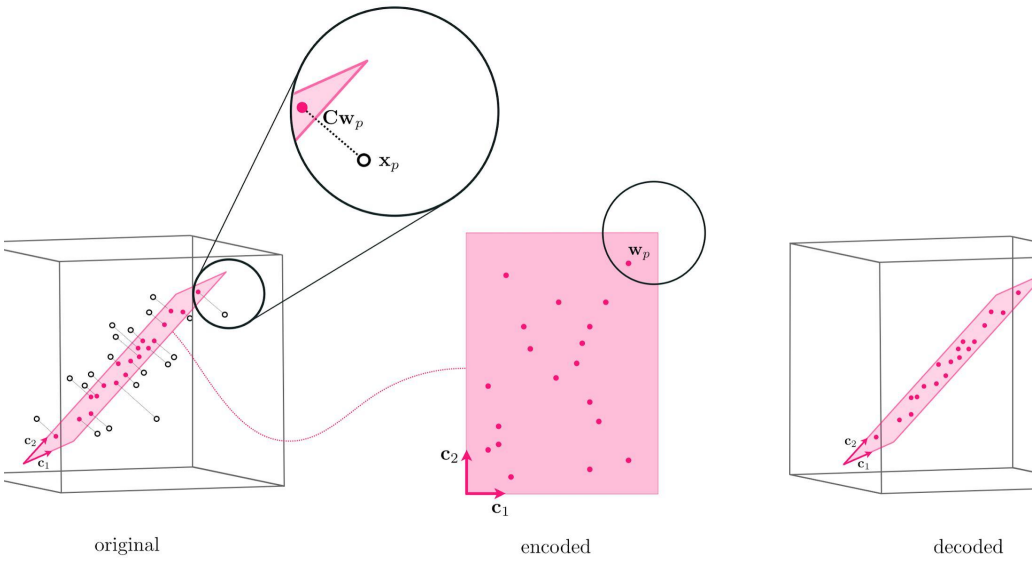


Figure 28. Encoding and decoding in a lower-dimensional subspace. The decoded point lies in the chosen lower-dimensional subspace and approximates the original point.

If the K subspace basis vectors are orthonormal, the encoding again simplifies to

$$\boldsymbol{w}_i = C^\top \boldsymbol{x}_i, \qquad i = 1, \ldots, n,$$

and decoding gives the orthogonal projection

$$CC^\top \boldsymbol{x}_i \approx \boldsymbol{x}_i, \qquad i = 1, \ldots, n.$$

The approximation becomes equality only when the original point already lies in the spanned subspace (or when the orthonormal set is a complete basis of the whole space).

Notation. For an orthonormal $d \times K$ matrix C, $C^\top \boldsymbol{x}_i$ is the K-dimensional encoding and $CC^\top \boldsymbol{x}_i$ is the decoded projection in the original d-dimensional space. The residual $\|CC^\top \boldsymbol{x}_i - \boldsymbol{x}_i\|_2$ measures the projection/reconstruction error between the original point and its projection.

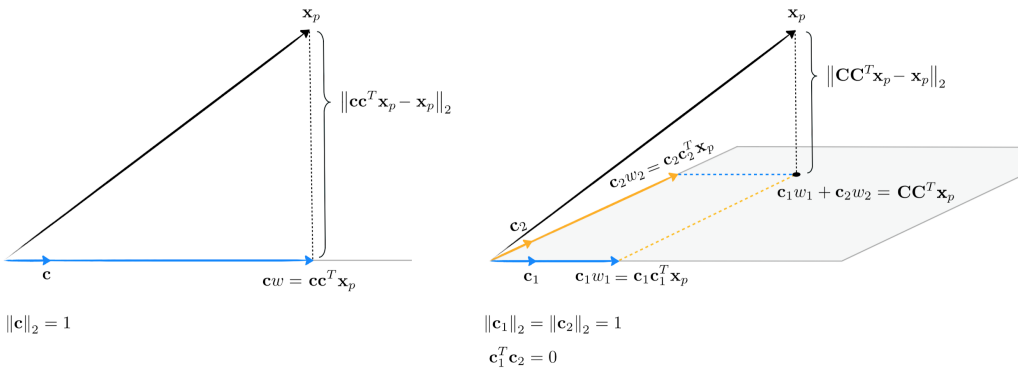


Figure 29. Geometric interpretation of orthonormal projection for one and two basis directions. The residual is perpendicular to the spanned subspace.

Vector Spaces and Projections — Key Takeaways

- A supervised dataset pairs feature vectors with targets; categorical features often require numerical encoding.
- Samples are modeled as independent, identically distributed draws from an unknown joint distribution P_{XY}.
- A spanning set generates a space through linear combinations; a basis is a minimal linearly independent spanning set.
- Orthonormal bases make encoding especially simple: coordinates are inner products, $\boldsymbol{w} = C^\top \boldsymbol{x}$.
- A non-standard basis changes coordinates but can still represent the same point exactly when it spans the entire space.
- With only $K < d$ directions, decoding produces a projection onto a lower-dimensional subspace rather than an exact reconstruction in general.

Part III

Optimization and Automatic Differentiation

17 Optimization Fundamentals

Optimization searches for a parameter vector $\boldsymbol{w}$ that minimizes or maximizes an **objective function** $g(\boldsymbol{w})$. When the objective is minimized in a learning problem, it is often called a **loss** or **cost function**. An optimizer is commonly denoted by $\boldsymbol{w}^\star$.

17.1 Global and local optima

A point $\boldsymbol{w}^\star$ is a global minimizer when

$$g(\boldsymbol{w}^\star) \le g(\boldsymbol{w}) \qquad \text{for every admissible } \boldsymbol{w}, \tag{4}$$

and a global maximizer when the inequality is reversed. Minimization and maximization are directly related:

$$\arg\max_{\boldsymbol{w}} g(\boldsymbol{w}) = \arg\min_{\boldsymbol{w}} [-g(\boldsymbol{w})].$$

A **local minimum** is smaller than or equal to the function values in a neighborhood of the point, but need not be the smallest value over the entire domain. Nonconvex objectives can therefore contain several local minima and maxima in addition to a global optimum.

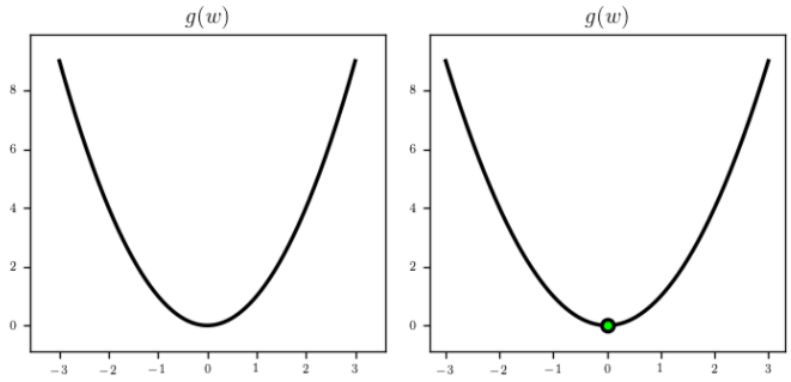


Figure 30. The strictly convex quadratic shown has a unique global minimum. The highlighted point marks the minimizer of the objective.

17.2 Zero-order and first-order conditions

A zero-order definition compares objective values directly, as in equation (4). First-order conditions use derivatives. For a differentiable scalar function $g(w)$, every differentiable local optimum in the interior of the domain must satisfy

$$\frac{d}{dw} g(w^\star) = 0.$$

In d dimensions the corresponding stationary-point condition is

$$\nabla g(\boldsymbol{w}^\star) = \mathbf{0}, \qquad \nabla g(\boldsymbol{w}) = \begin{bmatrix} \partial g/\partial w_1 \\ \partial g/\partial w_2 \\ \vdots \\ \partial g/\partial w_d \end{bmatrix}.$$

Stationarity is necessary, not sufficient

A zero gradient identifies a *stationary point*, not automatically a minimum. For example, $g(w) = w^3$ satisfies $g'(0) = 0$, yet $w = 0$ is neither a local minimum nor a local maximum. Additional information is needed to classify stationary points. Second-order information (the second derivative in one dimension or the Hessian in several dimensions) can help with this classification, although the optimization development below focuses mainly on first-order methods.

In simple problems, the stationary equations can sometimes be solved analytically in closed form. For more complicated objectives, a closed-form solution may be unavailable or algebraically impractical, which motivates iterative procedures such as gradient descent.

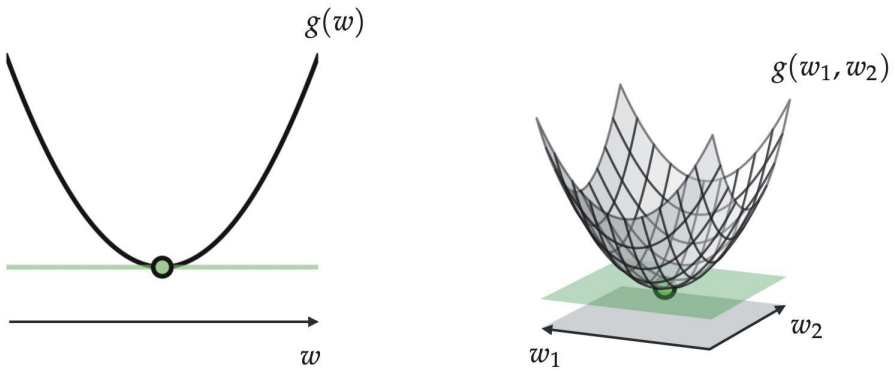


Figure 31. At a differentiable minimum, the first-order Taylor approximation is flat: a horizontal tangent in one dimension and a horizontal tangent hyperplane in higher dimensions.

18 Local Linear Approximation and the Gradient

For a differentiable function $g : \mathbb{R}^d \to \mathbb{R}$, the first-order Taylor approximation around $\boldsymbol{w}^0$ is

$$h(\boldsymbol{w}) = g(\boldsymbol{w}^0) + \nabla g(\boldsymbol{w}^0)^\top (\boldsymbol{w} - \boldsymbol{w}^0).$$

Equivalently, $h(\boldsymbol{w}) = a + \boldsymbol{b}^\top \boldsymbol{w}$, where

$$\boldsymbol{b} = \nabla g(\boldsymbol{w}^0), \qquad a = g(\boldsymbol{w}^0) - \nabla g(\boldsymbol{w}^0)^\top \boldsymbol{w}^0.$$

The vector $\boldsymbol{b}$ determines the orientation of the tangent hyperplane; the scalar a shifts it so that it touches g at $\boldsymbol{w}^0$.

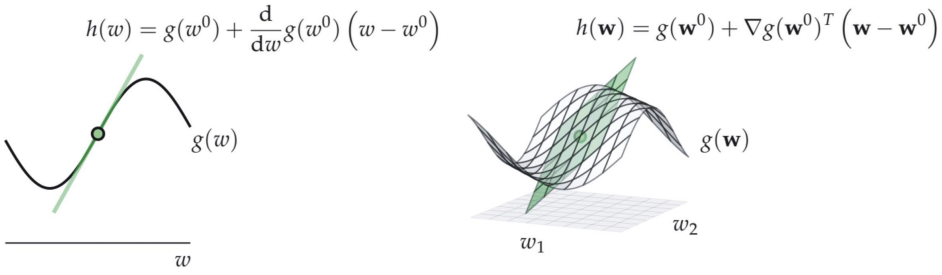


Figure 32. First-order Taylor approximation in one and two input dimensions. The tangent line or hyperplane matches the function locally at the expansion point.

18.1 Directions of steepest ascent and descent

Consider a linear function $h(\boldsymbol{w}) = a + \boldsymbol{b}^\top \boldsymbol{w}$ with $\boldsymbol{b} \neq \mathbf{0}$ and a unit direction $\boldsymbol{d}$, $\|\boldsymbol{d}\|_2 = 1$. With respect to the Euclidean norm, moving one unit from $\boldsymbol{w}^0$ changes the linear model by

$$h(\boldsymbol{w}^0 + \boldsymbol{d}) = a + \boldsymbol{b}^\top \boldsymbol{w}^0 + \boldsymbol{b}^\top \boldsymbol{d}.$$

Only $\boldsymbol{b}^\top \boldsymbol{d}$ depends on the direction. Since

$$\boldsymbol{b}^\top \boldsymbol{d} = \|\boldsymbol{b}\|_2 \|\boldsymbol{d}\|_2 \cos\theta = \|\boldsymbol{b}\|_2 \cos\theta,$$

the largest increase occurs when $\theta = 0$ and the largest decrease when $\theta = \pi$:

$$\boldsymbol{d}_{\text{asc}} = \frac{\boldsymbol{b}}{\|\boldsymbol{b}\|_2}, \qquad \boldsymbol{d}_{\text{desc}} = -\frac{\boldsymbol{b}}{\|\boldsymbol{b}\|_2}.$$

Applying this result to the local linear approximation, $\boldsymbol{b} = \nabla g(\boldsymbol{w}^0)$. Whenever $\nabla g(\boldsymbol{w}^0) \neq \mathbf{0}$, the normalized gradient gives the direction of steepest local ascent under the Euclidean norm, and its negative gives steepest local descent. At regular points, the gradient is normal to the corresponding smooth level curve or level surface.

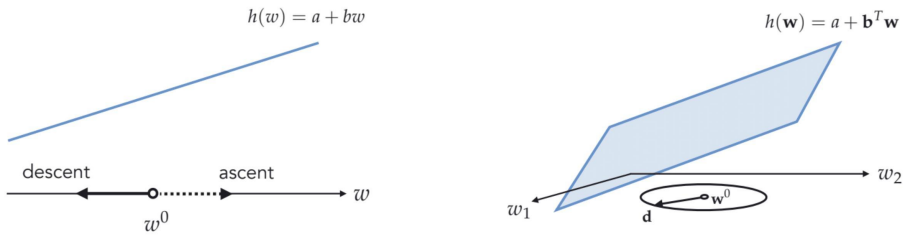


Figure 33. Ascent and descent directions for a linear function in one dimension and for a hyperplane in multiple dimensions.

Gradient — Geometric Meaning

At a point $\boldsymbol{w}^0$ with nonzero gradient, $\nabla g(\boldsymbol{w}^0)$ combines three pieces of local information: its direction gives steepest ascent under the Euclidean norm, its negative gives steepest descent, and its norm measures the first-order rate of increase in that steepest direction.

19 Computing Derivatives

Analytical differentiation can become cumbersome for deeply nested functions. A centered finite-difference approximation estimates a one-dimensional derivative at w^0 using a small ϵ:

$$\left.\frac{dg}{dw}\right|_{w^0} \approx \frac{g(w^0+\epsilon) - g(w^0-\epsilon)}{2\epsilon}.$$

This is a numerical approximation, whereas automatic differentiation computes derivatives by systematically applying differentiation rules to the operations that define the function.

Two essential rules are

$$\frac{d}{dw} f(g(w)) = f'(g(w))\, g'(w), \qquad \text{(chain rule)}$$

$$\frac{d}{dw}[f(w)g(w)] = f'(w)g(w) + f(w)g'(w). \qquad \text{(product rule)}$$

19.1 Computation graphs

A computation graph decomposes a function into elementary operations. For

$$g(w) = \tanh(w)\cos(w) + \log(w), \qquad w > 0,$$

where the restriction $w > 0$ ensures that the logarithm is real-valued. One convenient decomposition is

$$a = \tanh(w), \qquad b = \cos(w), \qquad c = \log(w), \qquad d = ab, \qquad e = d + c = g(w).$$

Each node is a function of its parent nodes, and evaluation proceeds from the input toward the final output.

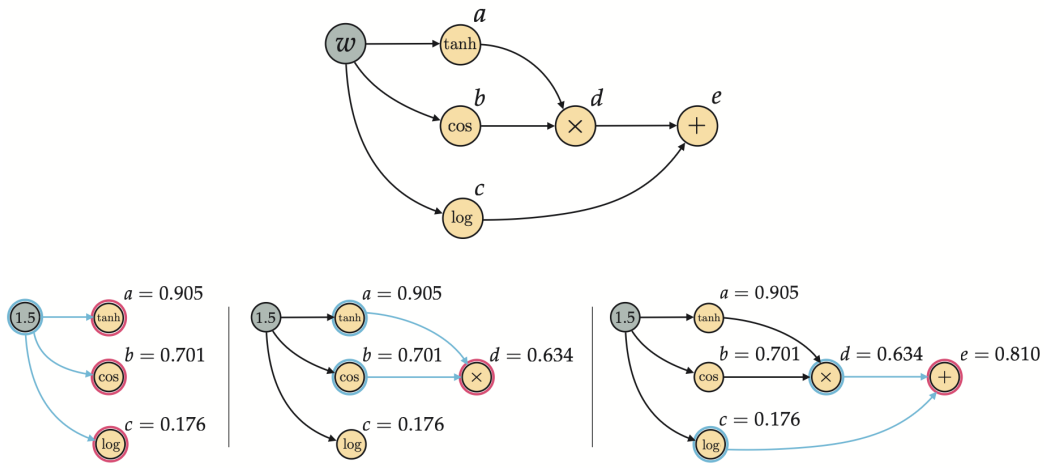


Figure 34. Computation graph for $g(w) = \tanh(w)\cos(w) + \log(w)$, together with a forward evaluation through its elementary operations.

19.2 Forward-mode automatic differentiation

Forward mode propagates both function values and derivatives from the input toward the output. Starting from $dw/dw = 1$,

$$\frac{da}{dw} = 1 - \tanh^2(w), \qquad \frac{db}{dw} = -\sin(w), \qquad \frac{dc}{dw} = \frac{1}{w}.$$

For $d = ab$, the product rule gives

$$\frac{dd}{dw} = \frac{da}{dw}b + a\frac{db}{dw} = (1 - \tanh^2 w)\cos w - \tanh w \sin w,$$

and therefore

$$\frac{dg}{dw} = (1 - \tanh^2 w)\cos w - \tanh w \sin w + \frac{1}{w}.$$

Forward mode is direct and exact up to floating-point arithmetic. More generally, one forward sweep propagates derivative information for one chosen input/tangent direction (a Jacobian–vector product). For a scalar-valued function of many independent inputs, constructing the entire gradient by forward mode generally requires one seed direction per input, so its cost grows with the input dimension.

19.3 Reverse mode and backpropagation

Reverse mode first evaluates the graph forward, then propagates derivative information backward from the scalar output toward the inputs. Along each path, local derivatives are multiplied by the chain rule; contributions from multiple child paths are summed.

For the two-input quadratic

$$g(w_1, w_2) = w_1^2 + w_2^2,$$

reverse accumulation recovers

$$\nabla g(w_1, w_2) = \begin{bmatrix} 2w_1 \\ 2w_2 \end{bmatrix}.$$

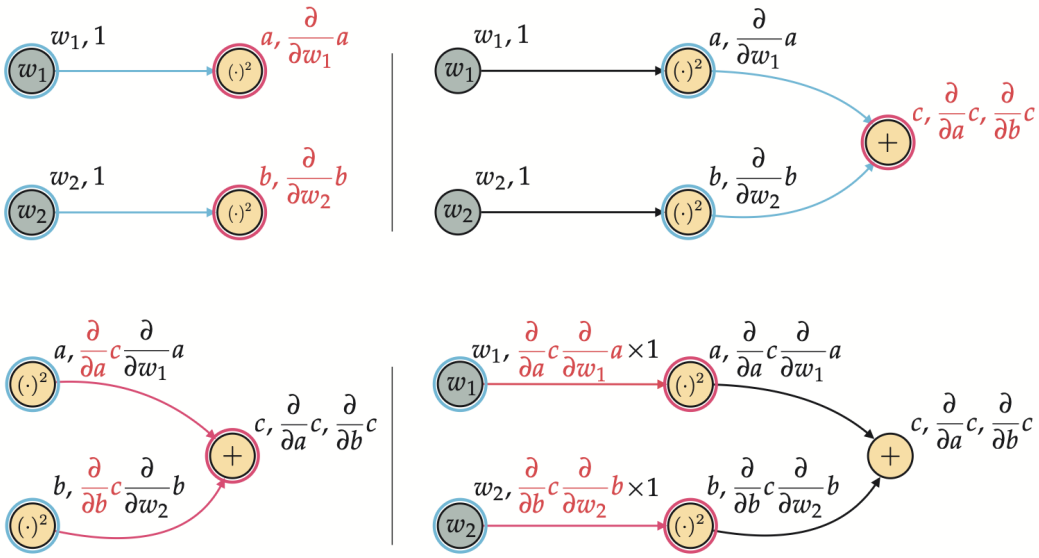


Figure 35. Reverse-mode differentiation on a two-input quadratic computation graph. Local derivatives are accumulated backward to obtain the gradient with respect to the inputs.

Reverse mode is particularly efficient for the common machine-learning setting of many parameters and one scalar loss: after the forward evaluation, one reverse sweep can accumulate the full gradient with respect to all inputs/parameters. It is the foundation of backpropagation and automatic-gradient systems used in modern deep-learning frameworks. Its main cost is that intermediate values from the forward pass must be retained for the backward sweep.

20 Gradient Descent

Gradient descent iteratively updates the parameters in the negative-gradient direction:

Gradient descent update

$$\boldsymbol{w}^{(k)} = \boldsymbol{w}^{(k-1)} - \alpha \nabla g\left(\boldsymbol{w}^{(k-1)}\right), \tag{5}$$

where $\alpha > 0$ is the **step size** or **learning rate**. When the gradient is nonzero, $-\nabla g$ is the steepest first-order descent direction under the Euclidean norm. A finite update decreases the objective only when the step size is appropriate for the local geometry and regularity of g.

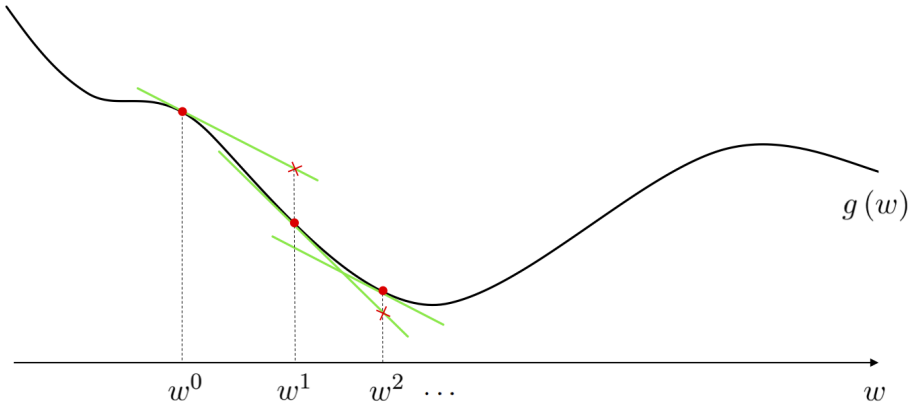


Figure 36. Gradient descent viewed through successive local tangent approximations. Each iterate moves in the negative-gradient direction before recomputing the local model.

20.1 Convex and nonconvex objectives

For a convex function, every local minimum is also a global minimum. For nonconvex objectives, different initializations can converge to different local minima or stall near stationary points. Consequently, both the initialization $\boldsymbol{w}^{(0)}$ and the step size α can materially affect the optimization trajectory.

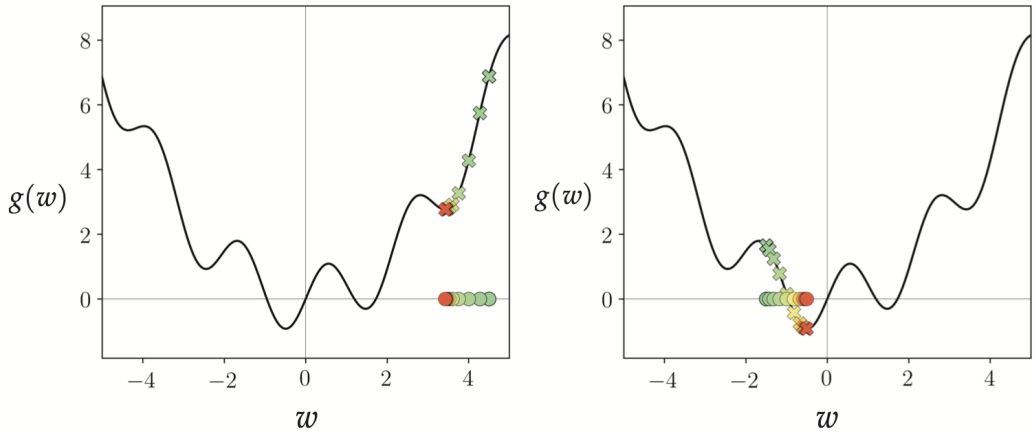


Figure 37. Gradient descent on a nonconvex one-dimensional objective from two different initializations. The trajectories can converge to different minima.

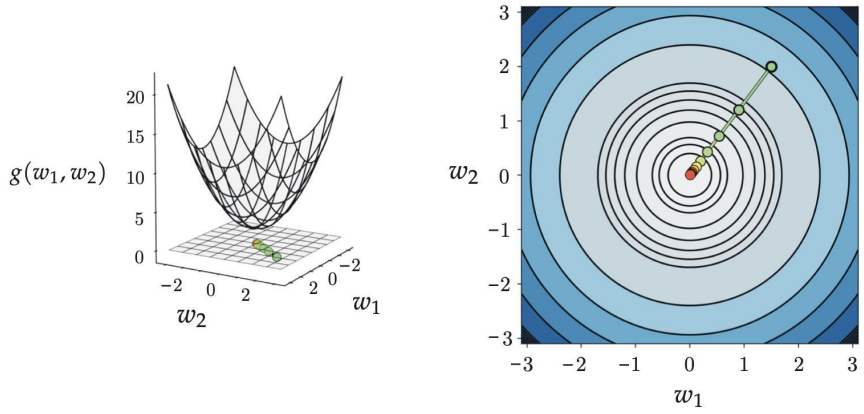


Figure 38. Gradient descent on a convex quadratic objective. The iterates move smoothly toward the unique global minimum.

20.2 Choosing the step size

The step size controls how far each update moves. A value that is too small produces slow progress; a value that is too large can overshoot the minimum and may even diverge. Illustrative schedules include a fixed step size, such as $\alpha = 10^{-m}$, and a diminishing schedule such as $\alpha_k = 1/k$ for $k \geq 1$. Whether a schedule converges depends on properties of the objective and on the step-size assumptions; no single schedule is universally convergent.

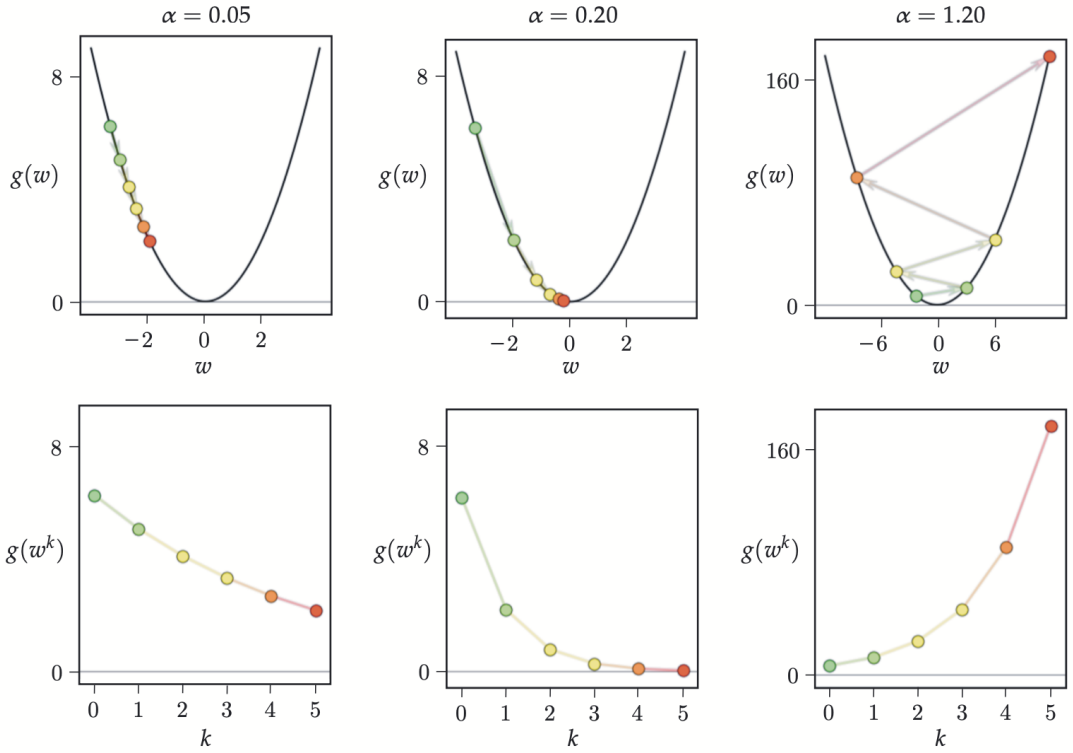


Figure 39. Effect of the step size on gradient descent for a quadratic objective: a small step converges slowly, a moderate step converges efficiently, and an excessively large step diverges.

For the nonsmooth objective $g(w) = |w|$, the illustrated subgradient update with a fixed step oscillates around the minimizer, while the illustrated diminishing schedule progressively reduces the oscillation toward zero.

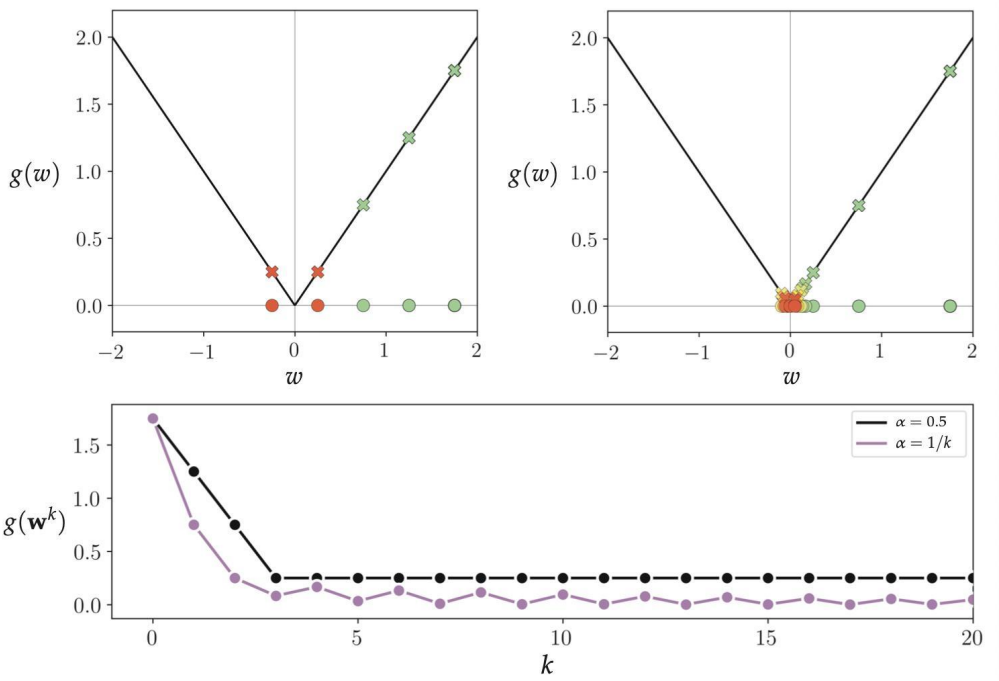


Figure 40. Fixed and diminishing step-length strategies for $g(w) = |w|$. The cost-history plot reveals persistent oscillation for the fixed step and convergence for the diminishing schedule.

20.3 Cost-function history plots

A cost-history plot displays $g(\boldsymbol{w}^{(k)})$ against iteration k. Such plots are useful for diagnosing divergence, oscillation, stagnation, and poorly chosen hyperparameters when the parameter space itself cannot be visualized directly. A useful trajectory need not decrease monotonically at every iteration; oscillatory behavior can still reach a lower final objective value.

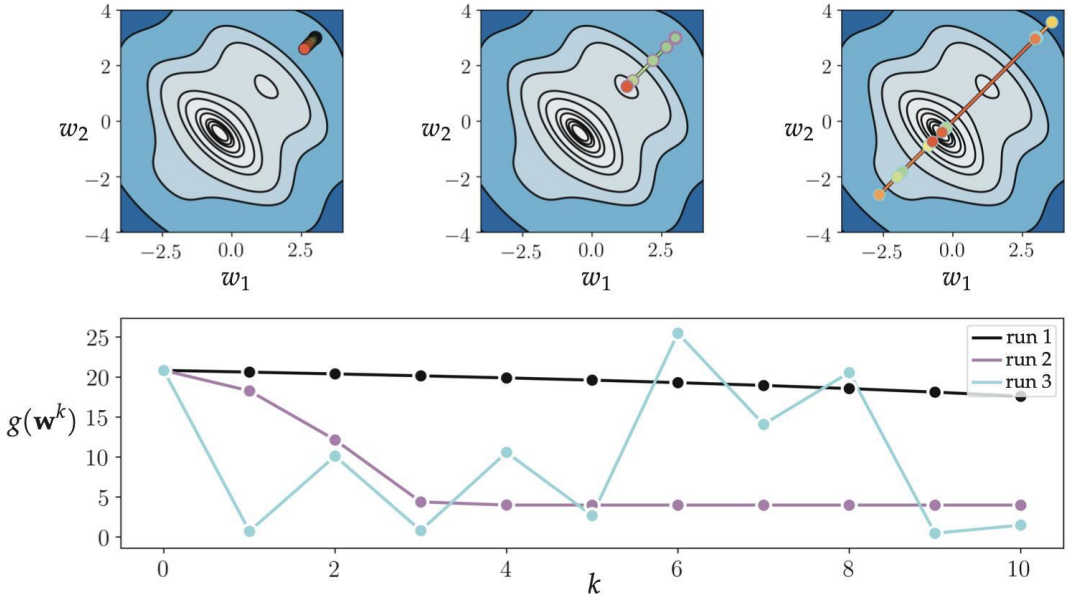


Figure 41. Three optimization trajectories on the same nonconvex objective and their cost histories. Different step sizes produce slow, locally trapped, or strongly oscillatory behavior.

20.4 Stopping criteria

Gradient descent is typically terminated when progress becomes negligible. Common criteria include

$$\begin{aligned} \|\nabla g(\boldsymbol{w}^{(k-1)})\|_2 &\leq \epsilon_g, \\ \|\boldsymbol{w}^{(k)} - \boldsymbol{w}^{(k-1)}\|_2 &\leq \epsilon_w, \\ \left|g(\boldsymbol{w}^{(k)}) - g(\boldsymbol{w}^{(k-1)})\right| &\leq \epsilon_f, \end{aligned}$$

or a prescribed maximum number of iterations. Optional normalizations by the parameter dimension merely rescale the corresponding tolerance and should be stated explicitly when used. Near a stationary point, $\nabla g \approx \mathbf{0}$, so the update in equation (5) becomes very small.

20.5 Weaknesses of vanilla gradient descent

Two characteristic difficulties are associated with gradient direction and magnitude:

- **Zig-zagging.** In a narrow elongated valley, gradients are nearly perpendicular to closely spaced contours, so successive steps alternate across the valley and convergence slows.
- **Slow crawling.** Near minima or saddle-type stationary regions, the gradient norm can become very small, producing tiny updates even when a better region exists elsewhere.

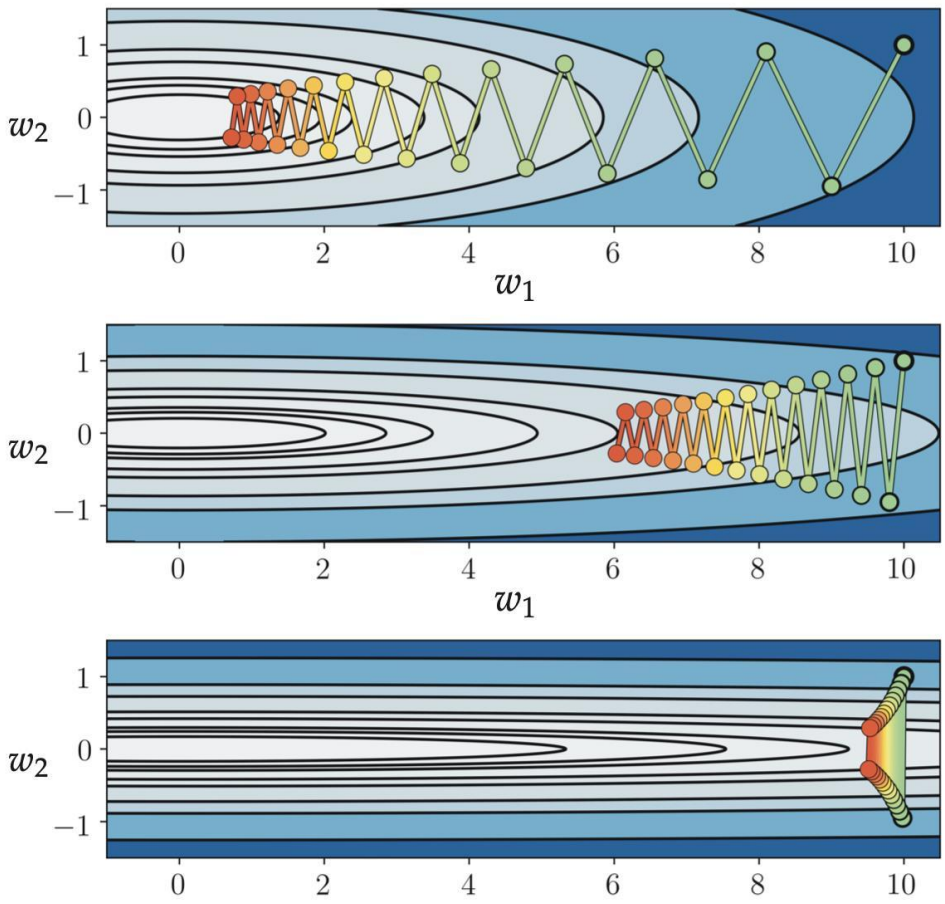


Figure 42. Increasing anisotropy in a quadratic objective produces progressively stronger zig-zagging under gradient descent.

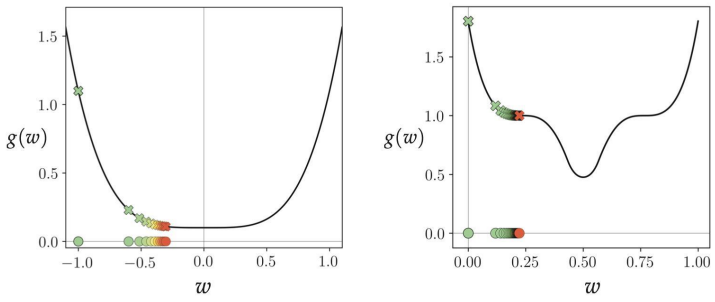


Figure 43. Small gradients cause slow progress near stationary regions; on a nonconvex objective, an iterate can effectively stall before reaching the best minimum.

20.6 Constrained gradient updates for PCA

Let $X \in \mathbb{R}^{n\times d}$ contain mean-centered observations as rows, and let $C \in \mathbb{R}^{d\times K}$ contain orthonormal directions, $C^\top C = I_K$. The variance-maximization objective is

$$\underset{C^\top C = I_K}{\text{maximize}} \ \frac{1}{n}\operatorname{tr}(C^\top X^\top X C) = \operatorname{tr}(C^\top \Sigma C), \qquad \Sigma = \frac{1}{n}X^\top X.$$

A Euclidean ascent step, absorbing $2/n$ into the step size, is

$$C^{(t+1/2)} = C^{(t)} + \alpha X^\top X C^{(t)},$$

followed by re-orthonormalizing the columns, for example using a thin QR factorization, to obtain $C^{(t+1)}$.

The equivalent reconstruction objective uses the row-wise reconstruction $XCC^\top$:

$$L(C) = \|XCC^\top - X\|_F^2, \qquad E(C) = XCC^\top - X.$$

Because C appears twice, its unrestricted Euclidean gradient has two terms:

$$\nabla_C L(C) = 2X^\top E(C)C + 2E(C)^\top XC. \tag{6}$$

On the constraint set, $E(C)C = X(CC^\top - I_d)C = 0$. Consequently, equation (6) simplifies to

$$\nabla_C L(C) = 2E(C)^\top XC = -2(I_d - CC^\top)X^\top XC.$$

This simplification requires orthonormal columns; use the full gradient for arbitrary C. Re-orthonormalize after each Euclidean update. Part IV derives the principal subspace as the constrained optimum.

Optimization and Automatic Differentiation — Key Takeaways

- First-order stationary points satisfy $\nabla g(\boldsymbol{w}) = \mathbf{0}$, but stationarity alone does not classify the point.
- The gradient is normal to level sets and gives the direction of steepest first-order ascent; $-\nabla g$ gives steepest descent.
- Computation graphs organize complex functions into elementary operations and make automatic differentiation systematic.
- Forward mode propagates derivatives from inputs to outputs; reverse mode propagates sensitivities from a scalar output back to many inputs.
- Gradient descent depends critically on step size, initialization, objective geometry, and stopping criteria.

Part IV

Principal Component Analysis and Dimensionality Reduction

21 Linear Transformations and Projection

For $\boldsymbol{x} \in \mathbb{R}^d$ and $A \in \mathbb{R}^{r \times d}$, the matrix-vector product

$$\boldsymbol{y} = A\boldsymbol{x} \in \mathbb{R}^r$$

defines a linear transformation. Different matrix structures have different geometric effects: diagonal matrices scale coordinate axes; square orthonormal matrices rotate or reflect without changing Euclidean lengths; off-diagonal structure can shear; and rank-deficient maps can collapse data into a lower-dimensional subspace.

Notation. In PCA, $X \in \mathbb{R}^{n \times d}$ stores mean-centered samples as rows. The matrix $C \in \mathbb{R}^{d \times K}$ contains K retained directions, $C^\top C = I_K$, and $\boldsymbol{z}_i = C^\top \boldsymbol{x}_i \in \mathbb{R}^K$ is the score vector for observation i. The complete score matrix is $XC \in \mathbb{R}^{n \times K}$ and its reconstruction is $XCC^\top \in \mathbb{R}^{n \times d}$.

For an orthonormal-column matrix $C \in \mathbb{R}^{d \times K}$, $C^\top C = I_K$. Encoding a point and decoding it are

$$\boldsymbol{w}_i = C^\top \boldsymbol{x}_i, \qquad C\boldsymbol{w}_i = CC^\top \boldsymbol{x}_i.$$

If C is a complete $d \times d$ orthonormal basis, $CC^\top \boldsymbol{x}_i = \boldsymbol{x}_i$. If $C \in \mathbb{R}^{d \times K}$ with $K < d$, the decoded vector is the orthogonal projection onto the K-dimensional subspace and generally differs from the original point.

22 From a Fixed Basis to a Learned Basis

With a fixed basis C, the encoding weights minimize the reconstruction error

$$h(\boldsymbol{w}_i) = \|C\boldsymbol{w}_i - \boldsymbol{x}_i\|_2^2.$$

Principal Component Analysis (PCA) extends this idea by learning the spanning directions themselves. The joint linear encoding-decoding objective can be written as

$$g(\boldsymbol{w}_1, \ldots, \boldsymbol{w}_n, C) = \frac{1}{n} \sum_{i=1}^{n} \|C\boldsymbol{w}_i - \boldsymbol{x}_i\|_2^2.$$

For an orthonormal C, the optimal encoding is $\boldsymbol{w}_i = C^\top \boldsymbol{x}_i$, yielding

$$g(C) = \frac{1}{n} \sum_{i=1}^{n} \|CC^\top \boldsymbol{x}_i - \boldsymbol{x}_i\|_2^2.$$

This is a linear autoencoder viewpoint: $C^\top$ encodes each sample and C decodes it back into the original space.

22.1 Reconstruction error and variance

For one mean-centered sample and orthonormal C,

$$\begin{aligned}\|CC^\top \boldsymbol{x}_i - \boldsymbol{x}_i\|_2^2 &= \boldsymbol{x}_i^\top CC^\top CC^\top \boldsymbol{x}_i - 2\boldsymbol{x}_i^\top CC^\top \boldsymbol{x}_i + \boldsymbol{x}_i^\top \boldsymbol{x}_i \\ &= -\|C^\top \boldsymbol{x}_i\|_2^2 + \|\boldsymbol{x}_i\|_2^2.\end{aligned}$$

Because $\|\boldsymbol{x}_i\|_2^2$ does not depend on C, minimizing reconstruction error is equivalent to maximizing the squared norm of the projected coordinates.

If $C = [\boldsymbol{c}_1 \ \cdots \ \boldsymbol{c}_K]$, then

$$\|C^\top \boldsymbol{x}_i\|_2^2 = \sum_{j=1}^{K} (\boldsymbol{c}_j^\top \boldsymbol{x}_i)^2.$$

Thus learning an orthonormal basis for the retained K-dimensional subspace is equivalent to maximizing projected variance along K mutually orthogonal directions.

23 Covariance and the First Principal Component

For a unit vector $\boldsymbol{c}_1$, define

$$h(\boldsymbol{c}_1) = \frac{1}{n} \sum_{i=1}^{n} (\boldsymbol{c}_1^\top \boldsymbol{x}_i)^2, \qquad \|\boldsymbol{c}_1\|_2 = 1.$$

When the data are mean-centered, this quantity is the **empirical variance under the $1/n$ normalization** along direction $\boldsymbol{c}_1$.

Stack the samples as

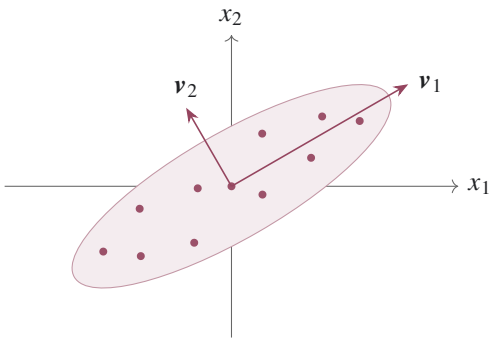
$$X = \begin{bmatrix} \boldsymbol{x}_1^\top \\ \boldsymbol{x}_2^\top \\ \vdots \\ \boldsymbol{x}_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}.$$

Then

Variance as a Rayleigh quotient

$$h(\boldsymbol{c}_1) = \frac{1}{n} \boldsymbol{c}_1^\top X^\top X \boldsymbol{c}_1 = \boldsymbol{c}_1^\top \Sigma \boldsymbol{c}_1, \qquad \Sigma = \frac{1}{n} X^\top X. \tag{7}$$

The matrix Σ is the empirical covariance matrix under the $1/n$ normalization. Its diagonal entries are empirical feature variances and its off-diagonal entries are empirical covariances between features. The alternative factor $1/(n-1)$ gives the usual unbiased sample-covariance estimator; changing between these two global normalizations rescales the eigenvalues but does not change the PCA eigenvector directions.



Principal directions are orthogonal;
$\lambda_1 > \lambda_2$ for this elongated cloud.

Figure 44. Geometric interpretation of a covariance matrix: orthogonal eigenvectors define the principal axes of an elongated data cloud, while the associated eigenvalues quantify spread along those axes.

24 Eigenvalues, Eigenvectors, and Principal Components

Because Σ is real and symmetric, it admits an orthonormal eigendecomposition

$$\Sigma \boldsymbol{v}_i = \lambda_i \boldsymbol{v}_i, \qquad \Sigma = V \Lambda V^\top, \qquad V^\top V = I.$$

For a covariance matrix, the eigenvalues are real and nonnegative. Ordering them as

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0,$$

the Rayleigh-quotient objective in equation (7) is maximized by any unit eigenvector associated with the largest eigenvalue. Choosing one such vector as $\boldsymbol{c}_1 = \boldsymbol{v}_1$ gives a **first principal-component direction**, and λ_1 is the empirical variance captured along that direction.

If an eigenvalue is repeated, individual eigenvectors inside its eigenspace are not unique. Changing the orthonormal basis preserves that full eigenspace. If the retention cutoff splits a group of equal eigenvalues, the optimal retained subspace itself need not be unique, although the captured variance is unchanged.

The second direction maximizes

$$h(\boldsymbol{c}_2) = \boldsymbol{c}_2^\top \Sigma \boldsymbol{c}_2$$

subject to $\|\boldsymbol{c}_2\|_2 = 1$ and $\boldsymbol{c}_1^\top \boldsymbol{c}_2 = 0$. Its solution is $\boldsymbol{c}_2 = \boldsymbol{v}_2$. Repeating this construction gives the ordered principal directions.

For a K-dimensional representation, PCA selects

$$C_K = [\boldsymbol{v}_1 \; \boldsymbol{v}_2 \; \cdots \; \boldsymbol{v}_K],$$

so that encoding and reconstruction become

PCA encoding and reconstruction

$$\boldsymbol{z}_i = C_K^\top \boldsymbol{x}_i, \qquad \hat{\boldsymbol{x}}_i = C_K \boldsymbol{z}_i = C_K C_K^\top \boldsymbol{x}_i. \tag{8}$$

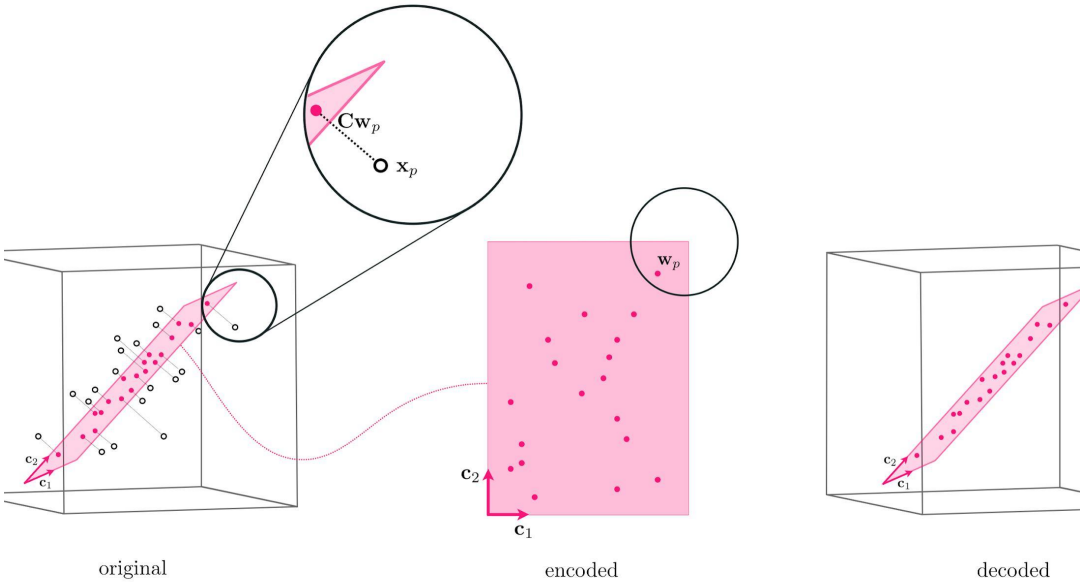


Figure 45. PCA with one retained component: original two-dimensional data are encoded to one coordinate and decoded onto the learned principal line.

> **PCA — Core Equivalence**
>
> For mean-centered data, minimizing linear reconstruction error over an orthonormal K-dimensional basis is equivalent to maximizing the variance captured by the projected coordinates. The global solution is given by the top K eigenvectors of the covariance matrix.

25 Dimensionality Reduction and Unsupervised Representation Learning

PCA is unsupervised: no target labels are required. It learns orthogonal directions that explain the strongest correlations and variability in the input distribution. Keeping only $K < d$ principal components maps the data into a lower-dimensional representation while retaining as much variance as possible within a linear subspace.

The transformed coordinates are uncorrelated with respect to the empirical covariance model, and the representation can expose redundancy and dominant directions of variation. In downstream workflows, these coordinates can be used as engineered features or as a compressed representation of the original data.

A common summary of retained information is the **explained-variance ratio**. For the first K principal directions,

$$\mathrm{EVR}(K) = \frac{\sum_{i=1}^{K} \lambda_i}{\sum_{i=1}^{d} \lambda_i}, \qquad \mathrm{EVR}(1) = \frac{\lambda_1}{\sum_{i=1}^{d} \lambda_i}. \tag{9}$$

This ratio requires positive total variance. If all centered observations are zero, the ratio is undefined and there is no nonzero variance to retain.

25.1 A compact PCA exercise

Exercise

Consider the two-dimensional samples

$$\boldsymbol{x}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \boldsymbol{x}_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \qquad \boldsymbol{x}_3 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Let $\bar{\boldsymbol{x}} = \frac{1}{3}\sum_{i=1}^{3} \boldsymbol{x}_i$ and denote the centered samples by $\tilde{\boldsymbol{x}}_i = \boldsymbol{x}_i - \bar{\boldsymbol{x}}$. Compute, in order: the sample mean and the centered samples; the empirical covariance matrix $\Sigma = \frac{1}{3}\sum_{i=1}^{3} \tilde{\boldsymbol{x}}_i \tilde{\boldsymbol{x}}_i^\top$; the eigenvalues and eigenvectors of Σ; the first and second principal directions; the one-dimensional principal-component scores; the one-dimensional reconstructions and their mean-squared reconstruction error; the explained-variance ratio of the first component; and the interpretation of the sign of the off-diagonal covariance entry.

25.2 Worked solution: from raw samples to reconstruction

1. Center the observations. The sample mean and centered data matrix are

$$\bar{\boldsymbol{x}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad X_c = \begin{bmatrix} (\boldsymbol{x}_1 - \bar{\boldsymbol{x}})^\top \\ (\boldsymbol{x}_2 - \bar{\boldsymbol{x}})^\top \\ (\boldsymbol{x}_3 - \bar{\boldsymbol{x}})^\top \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ -1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Each feature column sums to zero. Here the subscript c explicitly distinguishes centered data from the raw observations in the exercise.

2. Compute covariance. Under the $1/n = 1/3$ convention,

$$\Sigma = \frac{1}{3} X_c^\top X_c = \frac{1}{3} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Both features have variance $2/3$. Their positive covariance $1/3$ means that deviations from their means tend to have the same sign.

3. Find the principal directions. The characteristic equation is

$$\det(\Sigma - \lambda I_2) = \left(\frac{2}{3} - \lambda\right)^2 - \frac{1}{9} = 0.$$

Its ordered eigenpairs are

$$\lambda_1 = 1, \quad \boldsymbol{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \qquad \lambda_2 = \frac{1}{3}, \quad \boldsymbol{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

The directions are orthonormal. Reversing either eigenvector's sign reverses its scores but leaves reconstructions unchanged.

4. Encode with one component. Retain $C_1 = \boldsymbol{v}_1$. The score matrix is a single column:

$$\boldsymbol{z} = X_c \boldsymbol{v}_1 = \begin{bmatrix} \sqrt{2} \\ -1/\sqrt{2} \\ -1/\sqrt{2} \end{bmatrix}.$$

These are coordinates along the principal line, not predictions of a target label.

5. Decode and restore the original mean

The centered reconstructions are

$$\widehat{X}_c = \boldsymbol{z}\boldsymbol{v}_1^\top = \begin{bmatrix} 1 & 1 \\ -1/2 & -1/2 \\ -1/2 & -1/2 \end{bmatrix}.$$

Adding the mean to every reconstructed row gives reconstructions in the original feature coordinates:

$$\hat{\boldsymbol{x}}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \qquad \hat{\boldsymbol{x}}_2 = \begin{bmatrix} 1/2 \\ -1/2 \end{bmatrix}, \qquad \hat{\boldsymbol{x}}_3 = \begin{bmatrix} 1/2 \\ -1/2 \end{bmatrix}.$$

The first observation lies on the retained principal line, so it is reconstructed exactly. The other two project onto the same point because they have the same first-component score.

6. Check reconstruction error and retained variance. The squared Euclidean errors per observation are 0, $1/2$, and $1/2$. Hence

PCA exercise: two consistency checks

$$\frac{1}{3}\|X_c - \widehat{X}_c\|_F^2 = \frac{1}{3} = \lambda_2, \qquad \mathrm{EVR}(1) = \frac{1}{1 + 1/3} = \frac{3}{4} = 75\%.$$

Average squared reconstruction error equals the discarded variance, and the retained component explains 75% of total variance. If error is averaged over all six scalar entries instead of three observations, it is $1/6$; the normalization must be stated.

26 Linear Structure, Manifolds, and Charts

Linear dimensionality reduction works well when high-dimensional data lie close to a low-dimensional *linear* subspace. More general datasets may instead lie near a curved low-dimensional structure.

A **manifold** is a topological space that locally resembles Euclidean space. For a smooth one-dimensional manifold, a local **chart** is a coordinate map $\phi : U \to V \subset \mathbb{R}$ from a suitable region U of the manifold to an open Euclidean set V; its inverse $\phi^{-1} : V \to U$ is a local parameterization.

For example, on the open upper semicircle

$$U = \{(x, y) \in \mathbb{R}^2 : x^2 + y^2 = 1,\ y > 0\},$$

the map $\phi(x, y) = x$ is one-to-one onto $(-1, 1)$, with inverse parameterization

$$\phi^{-1}(t) = \left(t, \sqrt{1 - t^2}\right), \qquad t \in (-1, 1).$$

The restriction to a local/open arc is essential: a single coordinate such as x is not globally one-to-one on the entire circle.

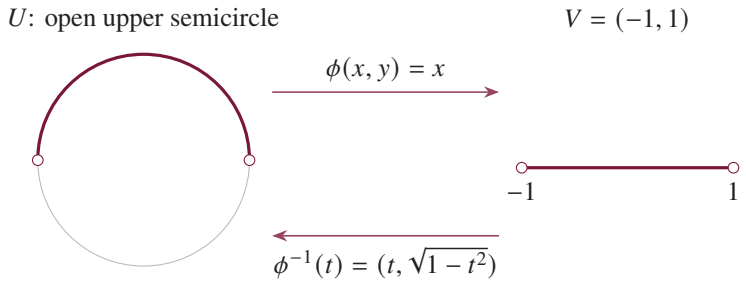


Figure 46. Local chart on the upper semicircle: ϕ maps a curved one-dimensional region to the Euclidean interval $(-1, 1)$, while ϕ^{-1} parameterizes the arc.

Principal Component Analysis — Key Takeaways

- PCA learns orthonormal principal directions; the retained vectors form a basis of the selected principal subspace (and a full basis of the ambient space only when $K = d$).
- For mean-centered data, reconstruction-error minimization is equivalent to variance maximization.
- The covariance eigenvectors give principal directions and the eigenvalues quantify variance along those directions.
- Retaining only the leading K eigenvectors gives a K-dimensional linear representation and an orthogonal reconstruction $C_K C_K^\top \boldsymbol{x}$.
- PCA is a linear method; curved low-dimensional structure motivates the broader language of manifolds and local charts.

Part V

Linear Models, Generalization, and Evaluation

27 Function Approximation and Risk Minimization

Supervised learning can be viewed as function approximation: from input–output examples, an algorithm chooses a predictor from a **hypothesis space** $\mathcal{F}$. The unknown relationship need not belong to this family, and observations can contain noise. Two design choices are therefore central: which functions are allowed, and which loss measures the quality of their predictions. The aim is to predict accurately on new observations, rather than merely reproduce the training labels.

Notation. In this part, $Z = ((\boldsymbol{x}_i, y_i))_{i=1}^n$ denotes n i.i.d. observations, and $\boldsymbol{x}_i \in \mathbb{R}^d$ contains d original input features. We add an intercept coordinate by defining $\tilde{\boldsymbol{x}}_i = (1, \boldsymbol{x}_i^\top)^\top \in \mathbb{R}^{d+1}$, with parameters $\boldsymbol{\beta} = (\beta_0, \dots, \beta_d)^\top$. To preserve the convention $X \in \mathbb{R}^{n\times d}$ for original features, write $\widetilde{X} = [\mathbf{1}\ X] \in \mathbb{R}^{n\times(d+1)}$ for the augmented design matrix; row i is $\tilde{\boldsymbol{x}}_i^\top$.

For a loss $\ell(f(\boldsymbol{x}), y)$, the **population risk** and its sample approximation, the **empirical risk**, are

$$R(f) = \mathbb{E}_{(\boldsymbol{x},y)\sim P_{XY}}[\ell(f(\boldsymbol{x}), y)],$$
$$\widehat{R}_Z(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(\boldsymbol{x}_i), y_i).$$

Since P_{XY} is unknown, empirical risk minimization (ERM) chooses

$$\hat{f} \in \arg\min_{f\in\mathcal{F}} \widehat{R}_Z(f).$$

A low training risk is useful only when the fitted predictor also has low population risk. Model capacity, sample size, regularization, and the evaluation procedure all affect this generalization.

28 Ordinary Least-Squares Regression

28.1 Model, residuals, and squared loss

An affine predictor is conventionally called a linear regression model:

$$f_{\boldsymbol{\beta}}(\boldsymbol{x}) = \beta_0 + \sum_{j=1}^{d} \beta_j x_j = \boldsymbol{\beta}^\top \tilde{\boldsymbol{x}}.$$

The intercept β_0 captures a constant offset. With targets $\boldsymbol{y} = (y_1, \dots, y_n)^\top$, predictions are $\hat{\boldsymbol{y}} = \widetilde{X}\boldsymbol{\beta}$ and residuals are $r_i = y_i - \hat{y}_i$. The squared objective is unchanged by this sign convention. Ordinary least squares (OLS) minimizes the mean squared error:

Least-squares objective

$$J(\boldsymbol{\beta}) = \frac{1}{n}\sum_{i=1}^{n} (\boldsymbol{\beta}^\top \tilde{\boldsymbol{x}}_i - y_i)^2 = \frac{1}{n}\|\widetilde{X}\boldsymbol{\beta} - \boldsymbol{y}\|_2^2. \tag{10}$$

Squaring prevents positive and negative residuals from canceling and gives large errors more weight. It also makes the fit sensitive to extreme target values. Multiplying this objective by a positive constant, such as $1/2$, changes its gradient scale but not its minimizers.

28.2 Derivatives and gradient updates

Differentiating with respect to coefficient β_j, including $j = 0$ with $\tilde{x}_{i0} = 1$, gives

$$\frac{\partial J}{\partial \beta_j} = \frac{2}{n} \sum_{i=1}^{n} (\boldsymbol{\beta}^\top \tilde{\boldsymbol{x}}_i - y_i) \tilde{x}_{ij},$$

$$\nabla J(\boldsymbol{\beta}) = \frac{2}{n} \widetilde{X}^\top (\widetilde{X}\boldsymbol{\beta} - \boldsymbol{y}). \tag{11}$$

Thus a batch gradient-descent step is

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \alpha \frac{2}{n} \widetilde{X}^\top (\widetilde{X}\boldsymbol{\beta}^{(t)} - \boldsymbol{y}).$$

For one input feature, this adds a multiple of the mean residual to the intercept and a multiple of the mean residual times input to the slope. All coordinates are updated from the same current parameter vector.

The Hessian is $\nabla^2 J = 2\widetilde{X}^\top \widetilde{X}/n$, a positive semidefinite matrix. Hence OLS is convex, and every stationary point is a global minimizer. For nonzero $\widetilde{X}$, a constant learning rate $0 < \alpha < 2/L$, where $L = 2\lambda_{\max}(\widetilde{X}^\top \widetilde{X})/n$, gives convergence of the parameter iterates to a minimizer. Poorly scaled features can make convergence slow; scaling should be learned from training data.

Table 5. Gradient estimators for squared-error regression.

Method	Examples per update	Main tradeoff
Batch GD	All n training examples	Exact empirical gradient; expensive per update on large datasets
Stochastic GD	One randomly sampled example	Cheap updates with high gradient noise
Mini-batch GD	A random subset B	Efficient vectorization with less noise than single-example updates

For a uniformly sampled batch B, the estimator

$$\boldsymbol{g}_B(\boldsymbol{\beta}) = \frac{2}{|B|} \sum_{i \in B} (\boldsymbol{\beta}^\top \tilde{\boldsymbol{x}}_i - y_i) \tilde{\boldsymbol{x}}_i$$

is unbiased for the full gradient. With a fixed step size, noisy methods can fluctuate near a minimizer; reducing the step size can improve convergence. Noise may help in nonconvex problems, but OLS has no nonglobal local minima to escape.

28.3 Normal equations and the geometry of the solution

Setting equation (11) to zero yields the **normal equations**:

$$\widetilde{X}^\top \widetilde{X} \hat{\boldsymbol{\beta}} = \widetilde{X}^\top \boldsymbol{y}. \tag{12}$$

When $\widetilde{X}$ has full column rank, $\widetilde{X}^\top \widetilde{X}$ is invertible and the unique solution is

$$\hat{\boldsymbol{\beta}} = (\widetilde{X}^\top \widetilde{X})^{-1} \widetilde{X}^\top \boldsymbol{y}. \tag{13}$$

The fitted vector $\widetilde{X}\hat{\boldsymbol{\beta}}$ is the orthogonal projection of $\boldsymbol{y}$ onto the column space of $\widetilde{X}$: the residual is orthogonal to every column because $\widetilde{X}^\top(\widetilde{X}\hat{\boldsymbol{\beta}} - \boldsymbol{y}) = \mathbf{0}$.

Full column rank requires $n \geq d + 1$ and no exact linear dependence among the columns. If rank is deficient, coefficient minimizers are nonunique, but the fitted vector is still unique. The Moore–Penrose pseudoinverse gives the minimum-norm minimizer $\hat{\boldsymbol{\beta}} = \widetilde{X}^+ \boldsymbol{y}$.

Closed form, convexity, and numerical computation

The normal equations arise from the quadratic least-squares objective; convexity alone does not guarantee a closed-form solution. Forming $\widetilde{X}^\top \widetilde{X}$ costs approximately $O(n(d+1)^2)$ and a dense factorization costs $O((d+1)^3)$. In computation, solve a linear system or use QR/SVD rather than explicitly forming an inverse. Nearly dependent features make normal-equation calculations particularly sensitive to numerical error.

A small regression fit

For $(x_i, y_i) = (0, 1), (1, 2), (2, 2)$,

$$\widetilde{X} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \qquad \widetilde{X}^\top \widetilde{X} = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}, \qquad \widetilde{X}^\top \boldsymbol{y} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}.$$

The normal equations give $\hat{\beta}_0 = 7/6$ and $\hat{\beta}_1 = 1/2$. Predictions are $7/6, 5/3, 13/6$, and the training MSE is $1/18$. The line balances residuals instead of passing through every observation.

28.4 Regression evaluation metrics

On an evaluation set of m observations, let $e_i = \hat{y}_i - y_i$ and $\bar{y} = m^{-1} \sum_i y_i$.

$$\mathrm{MSE} = \frac{1}{m} \sum_i e_i^2, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{m} \sum_i e_i^2},$$

$$\mathrm{MAE} = \frac{1}{m} \sum_i |e_i|, \qquad \mathrm{MRE} = \frac{1}{m} \sum_i \frac{|e_i|}{|y_i|},$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

RMSE and MAE have the target's units, while MSE has squared units. Here MRE means *mean absolute relative error*; multiplying it by 100% gives MAPE. It is undefined for zero targets and can be dominated by targets near zero. $R^2 = 1$ indicates perfect predictions; $R^2 = 0$ matches the evaluation-set mean baseline, and negative values are possible. R^2 is undefined when all evaluation targets are identical. For all four error measures, smaller is better; for R^2, larger is better.

29 Feature Maps and Model Capacity

29.1 Linear parameters, nonlinear input relationships

A feature map $\boldsymbol{\phi} : \mathbb{R}^d \to \mathbb{R}^q$ changes the representation before applying a linear predictor. In this model, redefine the parameter vector as $\boldsymbol{\beta} \in \mathbb{R}^q$:

$$f_{\boldsymbol{\beta}}(\boldsymbol{x}) = \boldsymbol{\beta}^\top \boldsymbol{\phi}(\boldsymbol{x}).$$

For a scalar input, $\boldsymbol{\phi}(x) = (1, x, x^2)^\top$ gives quadratic regression. For two inputs, a quadratic expansion is

$$\boldsymbol{\phi}(\boldsymbol{x}) = (1, x_1, x_2, x_1^2, x_1 x_2, x_2^2)^\top.$$

The interaction $x_1 x_2$ allows the effect of one input to depend on the other. The predictor can be nonlinear in its original inputs while remaining linear in its fitted coefficients, so the same least-squares machinery applies using the feature matrix $\Phi \in \mathbb{R}^{n \times q}$ with row i equal to $\boldsymbol{\phi}(\boldsymbol{x}_i)^\top$.

29.2 Underfitting and overfitting

Capacity describes the complexity of relationships a model family can represent. A low-capacity family may **underfit**, producing large errors on both training and new data. A highly flexible family may **overfit**, producing small training error but larger error on new data because it fits incidental noise.

For nested polynomial families, the minimum training MSE cannot increase as degree grows, but held-out error need not decrease. Feature choice therefore encodes assumptions about the relationship and must be evaluated on data outside the fit. Increasing the number of features also increases data and computation requirements; there is no universal sample-count formula that guarantees generalization for every distribution and model family.

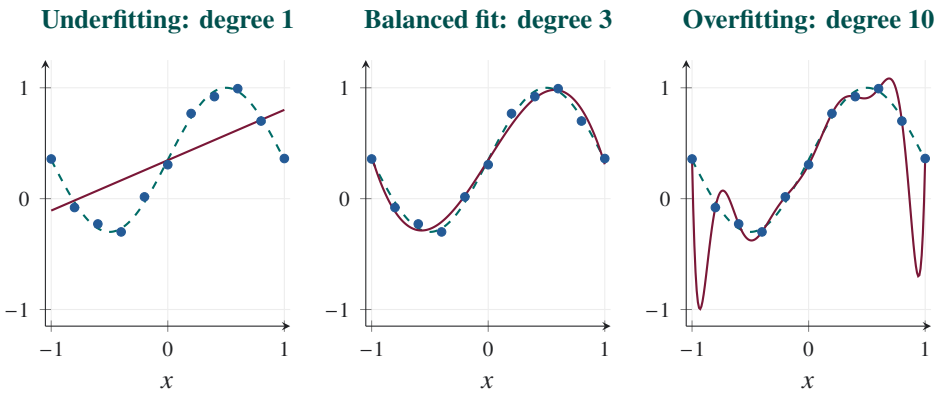


Figure 47. The same noisy observations fitted with polynomial degrees 1, 3, and 10. Blue points are observations, the dashed teal curve is the generating function, and the burgundy curve is the fitted model. The degree-1 fit misses the curved trend; degree 3 captures it; degree 10 interpolates all observations but oscillates between them. This illustrates capacity, rather than a universal rule that a particular degree is best.

30 Maximum Likelihood and Probabilistic Prediction

30.1 Likelihood and negative log-likelihood

Instead of specifying only a predictor, specify a conditional probability model $p_{\boldsymbol{\beta}}(y \mid \boldsymbol{x})$. For independent observations, its conditional likelihood and negative log-likelihood (NLL) are

$$\mathcal{L}(\boldsymbol{\beta}; Z) = \prod_{i=1}^{n} p_{\boldsymbol{\beta}}(y_i \mid \boldsymbol{x}_i),$$
$$\mathrm{NLL}(\boldsymbol{\beta}; Z) = -\sum_{i=1}^{n} \log p_{\boldsymbol{\beta}}(y_i \mid \boldsymbol{x}_i).$$

Maximum likelihood estimation maximizes $\mathcal{L}$, equivalently minimizes NLL. Logs turn a product of small values into a sum and simplify differentiation. Dividing NLL by n preserves the optimum; the scaling must be accounted for when adding regularization.

30.2 Gaussian noise recovers least squares

Suppose

$$y_i = \boldsymbol{\beta}^\top \tilde{\boldsymbol{x}}_i + \varepsilon_i, \qquad \varepsilon_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2),$$

with independent noise and fixed $\sigma^2 > 0$. Then

$$\mathrm{NLL}(\boldsymbol{\beta}; Z) = \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_{i=1}^{n} (y_i - \boldsymbol{\beta}^\top \tilde{\boldsymbol{x}}_i)^2.$$

The first term is constant in $\boldsymbol{\beta}$, so Gaussian maximum likelihood gives the OLS coefficients. The conditional Gaussian's mean and mode both equal $\boldsymbol{\beta}^\top \tilde{\boldsymbol{x}}$, recovering the regression prediction. Thus choosing a loss can be understood as choosing a probabilistic observation model: squared error corresponds to Gaussian noise, while classification requires a distribution over discrete labels.

31 Binary Logistic Regression

31.1 Scores, probabilities, and the decision boundary

For $y \in \{0, 1\}$, let $\boldsymbol{w} = (\beta_1, \ldots, \beta_d)^\top$ and define the score $s(\boldsymbol{x}) = \beta_0 + \boldsymbol{w}^\top \boldsymbol{x} = \boldsymbol{\beta}^\top \tilde{\boldsymbol{x}}$. Normalizing the symmetric exponential weights gives

$$p_{\boldsymbol{\beta}}(Y = 1 \mid \boldsymbol{x}) = \frac{e^{s/2}}{e^{-s/2} + e^{s/2}} = \frac{1}{1 + e^{-s}} = \sigma(s), \qquad p_{\boldsymbol{\beta}}(Y = 0 \mid \boldsymbol{x}) = 1 - \sigma(s).$$

The sigmoid maps a real score to $(0, 1)$, with $\sigma(0) = 1/2$ and derivative $\sigma'(s) = \sigma(s)(1 - \sigma(s))$. The score is the log-odds:

$$\log \frac{p_{\boldsymbol{\beta}}(Y = 1 \mid \boldsymbol{x})}{p_{\boldsymbol{\beta}}(Y = 0 \mid \boldsymbol{x})} = s(\boldsymbol{x}).$$

With threshold $1/2$, the prediction is $\hat{y} = \mathbb{1}\{s(\boldsymbol{x}) \geq 0\}$. For $\boldsymbol{w} \neq \boldsymbol{0}$, the boundary $\beta_0 + \boldsymbol{w}^\top \boldsymbol{x} = 0$ is an affine hyperplane with normal $\boldsymbol{w}$, and signed Euclidean distance is $s(\boldsymbol{x})/\|\boldsymbol{w}\|_2$. It passes through the origin only when $\beta_0 = 0$.

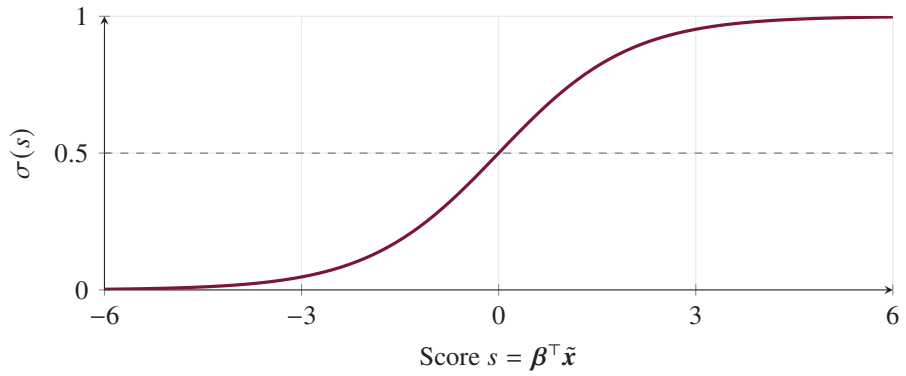


Figure 48. The sigmoid turns a linear score into a binary-class probability. A score of zero corresponds to the usual probability threshold of one half.

A threshold $\tau \in (0, 1)$ instead predicts class 1 when $s \geq \log(\tau/(1-\tau))$. Changing the threshold changes the balance between false positives and false negatives. In an energy interpretation $p \propto e^{-E}$, the above weights correspond to $E_0 = s/2$ and $E_1 = -s/2$: increasing s lowers the energy and increases the probability of class 1.

31.2 Bernoulli likelihood and binary cross-entropy

Writing $p_i = \sigma(\boldsymbol{\beta}^\top \tilde{\boldsymbol{x}}_i)$, the Bernoulli model is $p_{\boldsymbol{\beta}}(y_i \mid \boldsymbol{x}_i) = p_i^{y_i}(1-p_i)^{1-y_i}$. Its average NLL is the **binary cross-entropy**:

Binary cross-entropy

$$J_{\log}(\boldsymbol{\beta}) = -\frac{1}{n}\sum_{i=1}^{n}\big[y_i \log p_i + (1-y_i)\log(1-p_i)\big]. \tag{14}$$

For $y_i = 1$, the individual loss is $-\log p_i$; for $y_i = 0$, it is $-\log(1-p_i)$. Confident wrong predictions receive arbitrarily large penalties. The loss tends to zero as the probability of the true class tends to one.

31.3 Gradient and optimization

For one observation, use $\partial p_i/\partial \boldsymbol{\beta} = p_i(1-p_i)\tilde{\boldsymbol{x}}_i$:

$$\nabla_{\boldsymbol{\beta}} \ell_i = \left(-\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i}\right) p_i(1-p_i)\tilde{\boldsymbol{x}}_i = (p_i - y_i)\tilde{\boldsymbol{x}}_i.$$

Consequently,

$$\nabla J_{\log}(\boldsymbol{\beta}) = \frac{1}{n}\widetilde{X}^\top(\boldsymbol{p} - \boldsymbol{y}), \qquad \boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \alpha \nabla J_{\log}(\boldsymbol{\beta}^{(t)}). \tag{15}$$

This resembles the OLS update, with probabilities replacing linear predictions. The Hessian $\widetilde{X}^\top W \widetilde{X}/n$, where $W_{ii} = p_i(1-p_i)$, is positive semidefinite, so the objective is convex, but there is generally no closed-form coefficient solution. For strictly separable data, unregularized maximum likelihood may have no finite maximizer: coefficients can grow without bound while loss approaches zero.

For stable computation, express the individual loss as $\log(1 + e^{s_i}) - y_i s_i$ and evaluate its first term as $\max(0, s_i) + \log(1 + e^{-|s_i|})$.

32 Multiclass Logistic Regression and Softmax

For K classes, give class k a coefficient vector $\boldsymbol{\beta}_k$ and score $s_k = \boldsymbol{\beta}_k^\top \tilde{\boldsymbol{x}}$. The **softmax** model is

$$p_k(\boldsymbol{x}) = p(Y = k \mid \boldsymbol{x}) = \frac{e^{s_k}}{\sum_{j=1}^{K} e^{s_j}}, \qquad \hat{y} = \arg\max_k \, p_k(\boldsymbol{x}) = \arg\max_k \, s_k. \tag{16}$$

The probabilities are positive and sum to one. Adding the same scalar to every score leaves them unchanged, so the unconstrained parameterization is redundant. One can fix a reference class's coefficient vector to zero. With two classes, $p_1 = \sigma(s_1 - s_2)$, recovering binary logistic regression.

Using one-hot targets $y_{ik} = \mathbb{1}\{y_i = k\}$, the average categorical NLL, or **multiclass cross-entropy**, is

$$J_{\text{softmax}} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik} \log p_k(\boldsymbol{x}_i) = \frac{1}{n}\sum_{i=1}^{n}\left[\log \sum_{k=1}^{K} e^{s_{ik}} - s_{i,y_i}\right],$$

$$\nabla_{\boldsymbol{\beta}_k} J_{\text{softmax}} = \frac{1}{n}\sum_{i=1}^{n}\big(p_k(\boldsymbol{x}_i) - y_{ik}\big)\tilde{\boldsymbol{x}}_i.$$

Only the true class contributes to each example's target-weighted log-probability. Gradient updates increase its score relative to competing classes. Numerically, compute softmax using $e^{s_k - a}/\sum_j e^{s_j - a}$ with $a = \max_j s_j$ to avoid overflow; compute cross-entropy through the corresponding log-sum-exp expression.

33 Bias, Variance, and Expected Prediction Error

33.1 Repeated training sets

Imagine drawing many independent training sets Z from the same distribution and fitting a predictor $\hat{f}_Z$ to each. At a fixed input $\boldsymbol{x}$, **bias** measures the discrepancy between the average fitted prediction and the true conditional mean $f(\boldsymbol{x}) = \mathbb{E}[Y \mid \boldsymbol{x}]$. **Variance** measures how much the fitted prediction changes across training sets.

The lecture's example takes $f(x) = x + 2$, noise uniform on $[-1, 1]$, and repeatedly draws training sets of six observations. A constant model has limited ability to reproduce the trend, while a high-degree polynomial can change greatly with the sampled noise. A suitably chosen linear model can capture the relationship with less fluctuation. Here the noise variance is $1/3$.

33.2 Deriving the squared-error decomposition

Let a fresh test response satisfy $Y = f(\boldsymbol{x}) + \varepsilon$, with conditional mean-zero noise, variance $\sigma^2(\boldsymbol{x})$, and noise independent of the training set. Define $\bar{f}(\boldsymbol{x}) = \mathbb{E}_Z[\hat{f}_Z(\boldsymbol{x})]$. Then

$$Y - \hat{f}_Z(\boldsymbol{x}) = \big(f(\boldsymbol{x}) - \bar{f}(\boldsymbol{x})\big) + \big(\bar{f}(\boldsymbol{x}) - \hat{f}_Z(\boldsymbol{x})\big) + \varepsilon.$$

Squaring and averaging gives

Expected squared prediction error

$$\mathbb{E}_{Z,\varepsilon}\big[(Y - \hat{f}_Z(\boldsymbol{x}))^2 \mid \boldsymbol{x}\big] = \underbrace{(\bar{f}(\boldsymbol{x}) - f(\boldsymbol{x}))^2}_{\text{squared bias}} + \underbrace{\mathbb{E}_Z[(\hat{f}_Z(\boldsymbol{x}) - \bar{f}(\boldsymbol{x}))^2]}_{\text{variance}} + \underbrace{\sigma^2(\boldsymbol{x})}_{\text{irreducible noise}}. \tag{17}$$

The bias–variance cross term vanishes because $f - \bar{f}$ is fixed at this input and $\mathbb{E}_Z[\bar{f} - \hat{f}_Z] = 0$. The noise cross terms vanish because fresh test noise has zero conditional mean and is independent of training. Averaging equation (17) over test inputs gives the population squared-error decomposition.

Increasing capacity often decreases bias and increases variance, while more training data often reduces variance. These are useful tendencies, not universal monotonic laws. The decomposition concerns expected squared error over repeated training sets; it is not a decomposition of a single observed training error, and its terms are squared bias, variance, and noise.

34 Regularization: Ridge and Lasso

34.1 Penalizing complexity

Regularization expresses a preference among predictors by adding a coefficient penalty:

$$J_\lambda(\boldsymbol{\beta}) = \widehat{R}_Z(f_\beta) + \lambda \mathcal{R}(\boldsymbol{\beta}), \qquad \lambda \geq 0.$$

The hyperparameter λ is selected outside the coefficient-fitting procedure. Larger values emphasize the penalty, typically shrinking coefficients and reducing variance at the cost of increased bias. The intercept is usually excluded from this penalty. Feature scaling matters because penalties act on coefficient magnitudes; standardize using training-set statistics and reuse those statistics on held-out data.

34.2 Ridge regression and a Gaussian prior

With the mean-squared-error convention in equation (10), ridge regression minimizes

$$J_{\text{ridge}}(\boldsymbol{\beta}) = \frac{1}{n}\|\widetilde{X}\boldsymbol{\beta} - \boldsymbol{y}\|_2^2 + \lambda \sum_{j=1}^{d} \beta_j^2. \tag{18}$$

Let $D = \operatorname{diag}(0, 1, \ldots, 1)$ so that the intercept is unpenalized. Differentiation gives

Ridge normal equations

$$(\widetilde{X}^\top \widetilde{X} + n\lambda D)\hat{\boldsymbol{\beta}}_\lambda = \widetilde{X}^\top \boldsymbol{y}. \tag{19}$$

For $\lambda > 0$ and at least one observation, this system is positive definite with the intercept column of ones: every nonzero slope vector incurs a positive penalty, while a pure nonzero intercept gives nonzero fitted values. Ridge therefore gives unique coefficients even with dependent features. The factor $n\lambda$ follows from using *mean* squared error; a sum-of-squares objective would use λ instead.

Ridge also has a probabilistic interpretation. Under Gaussian observation noise and independent Gaussian priors $\beta_j \sim \mathcal{N}(0, \tau^2)$ on the slopes, maximum a posteriori (MAP) estimation minimizes Gaussian NLL plus $\sum_{j=1}^{d} \beta_j^2/(2\tau^2)$. Rescaling to equation (18) gives $\lambda = \sigma^2/(n\tau^2)$, with a flat prior on the intercept.

34.3 Lasso and sparsity

Lasso replaces the squared penalty by an absolute-value penalty:

$$J_{\text{lasso}}(\boldsymbol{\beta}) = \frac{1}{n}\|\widetilde{X}\boldsymbol{\beta} - \boldsymbol{y}\|_2^2 + \lambda \sum_{j=1}^{d} |\beta_j|. \tag{20}$$

Its corner at zero allows some fitted coefficients to be exactly zero, yielding a sparse model and implicit feature selection. Ridge usually shrinks coefficients without setting them exactly to zero. The lasso objective is convex but nondifferentiable at zero, so optimization uses methods such as coordinate descent or proximal updates rather than an ordinary derivative everywhere. An independent Laplace prior on slopes gives the corresponding MAP interpretation.

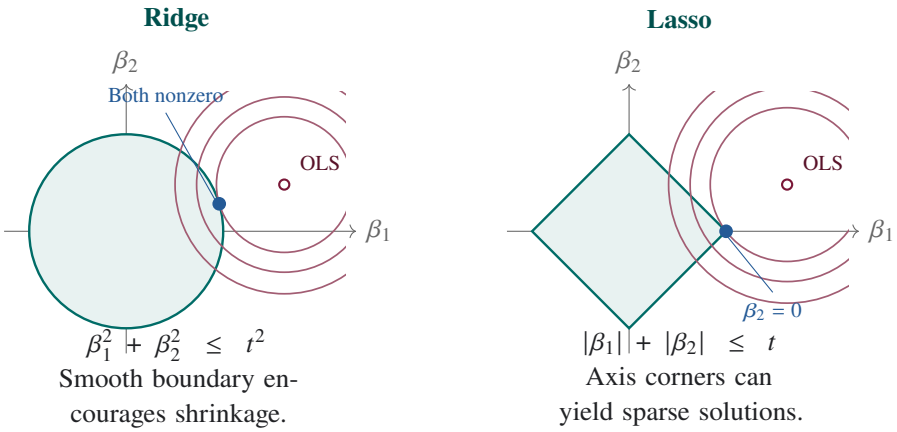


Figure 49. Constraint geometry for two slope coefficients. Teal regions represent an L_2 ball (ridge) and an L_1 ball (lasso); burgundy circles are contours of an illustrative squared-error loss centered at the unregularized optimum. The blue point is where the smallest attainable contour touches the constraint. The lasso diamond has corners on the coordinate axes, making an exactly zero coefficient possible. The correspondence between a constraint radius and a penalty strength depends on the problem.

35 Model Selection and Honest Evaluation

35.1 Training, validation, and test sets

A large gap between low training loss and higher held-out loss suggests overfitting. Compare *average* losses when the sets have different sizes. Repeatedly choosing hyperparameters using test results adapts the procedure to that test set and makes its reported performance optimistic.

1. **Split the data.** Use training data for fitting, validation data for selecting hyperparameters and feature maps, and test data for final evaluation. A 60/20/20 split is an illustration, not a requirement.
2. **Fit candidates on training data.** For each candidate, learn preprocessing and coefficients using only the training set.

3. **Select on validation data.** Compare a suitable metric and choose settings such as polynomial degree, λ, or a classification threshold.
4. **Fit the selected procedure.** Optionally refit on the combined training and validation data with the selected settings fixed, including refitting preprocessing on this combined set.
5. **Evaluate once on test data.** Report the selected metric on the untouched test set.

Random shuffling is appropriate for ordinary i.i.d. data; class-stratified splits can preserve class proportions. Temporal observations need chronological splits, and grouped observations need group-aware splits to avoid leakage between related samples.

35.2 k-fold cross-validation

Keep the final test set aside. Divide the remaining development data into k folds. For each hyperparameter candidate, train on $k - 1$ folds and validate on the remaining fold, rotating until each fold has been used for validation. Average the validation scores, select the candidate, and refit on all development data before the final test.

> **Prevent data leakage**
>
> Fit scaling, imputation, feature selection, and PCA using only each fold's training portion. Apply the fitted transformations to that fold's validation portion. Learning preprocessing from all observations before cross-validation leaks information into validation and can inflate the score.

Larger k uses more observations in each fit but costs more computation; leave-one-out uses one held-out observation per fold. Increasing k does not guarantee a uniformly more accurate model-selection estimate. Nested cross-validation separates inner hyperparameter tuning from outer performance estimation when a single held-out test set is impractical.

36 Classification Metrics and Decision Thresholds

36.1 The confusion matrix

For binary classification, identify the positive class explicitly. Rows below are actual labels and columns are predicted labels.

Table 6. Binary confusion matrix. False positives are false alarms; false negatives are missed positives.

	Predicted negative	Predicted positive
Actual negative	True negative (TN)	False positive (FP)
Actual positive	False negative (FN)	True positive (TP)

36.2 Accuracy, precision, recall, specificity, and F1

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, \qquad \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}},$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}, \qquad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}},$$

$$F_1 = \frac{2\ \text{Precision}\ \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}.$$

Recall is also sensitivity or the true positive rate (TPR), and specificity is the true negative rate; FPR = 1 − Specificity. **Precision** asks how many predicted positives are correct, while **recall** asks how many actual positives are found. F_1 is their harmonic mean and does not count true negatives. Fractions with zero denominators require an explicitly stated reporting convention.

Why accuracy can hide failure

If only one of 100 observations is positive, predicting every observation as negative gives TN = 99, FN = 1, and 99% accuracy, but zero recall. Precision is undefined because no positive predictions were made. For a second example, TP = 30, FP = 10, FN = 20, and TN = 40 give accuracy 0.70, precision 0.75, recall 0.60, specificity 0.80, and $F_1 = 2/3$.

Choose metrics according to the relative costs of missed positives and false alarms and the class balance. For multiclass tasks, report per-class metrics and specify averaging: macro averages give classes equal weight, while micro averages pool counts across classes.

36.3 ROC curves and area under the curve

A receiver operating characteristic (ROC) curve varies the score threshold and plots TPR vertically against FPR horizontally. Lowering the threshold generally identifies more positives and creates more false alarms. A perfect ranking permits the point $(0, 1)$; a random ranking has expected AUC 0.5.

ROC AUC summarizes ranking performance across thresholds. It equals the probability that a randomly chosen positive receives a higher score than a randomly chosen negative, counting ties as one half. AUC 1 means perfect separation of the evaluated classes. AUC below 0.5 can indicate reversed ranking; there is no universal AUC cutoff that makes a model useful for every task. AUC does not assess probability calibration or choose an operating threshold. With severe class imbalance, precision–recall curves also help reveal the precision achieved while recovering positives.

Linear Models — Key Takeaways

- ERM fits a hypothesis family to observed loss; generalization concerns expected performance on new data.
- OLS is a convex quadratic problem with normal equations; a unique coefficient solution requires full column rank.
- Feature maps allow nonlinear input relationships while preserving linear dependence on coefficients.
- Gaussian likelihood yields squared error; Bernoulli and categorical likelihoods yield logistic and softmax cross-entropy.
- Expected squared prediction error separates into squared bias, variance, and irreducible noise under the stated assumptions.
- Ridge shrinks coefficients and lasso can select a sparse set; validation or cross-validation selects their strength.
- Keep the final test set independent of selection, and choose metrics that reflect class balance and error costs.

Table 7. Losses and penalties at a glance. Average the observation losses over training examples, then add any coefficient penalty. Here $s_i = \boldsymbol{\beta}^\top \tilde{\boldsymbol{x}}_i$ and $r_i = y_i - s_i$ for linear regression; s_{ik} is a multiclass score.

Model / method	Loss or penalty	What to remember
Least squares	r_i^2	Squared residual; Gaussian likelihood interpretation
Binary logistic	$\log(1 + e^{s_i}) - y_i s_i$	Bernoulli negative log-likelihood
Multiclass logistic	$\log \sum_k e^{s_{ik}} - s_{i,y_i}$	Categorical negative log-likelihood
Ridge penalty	$\lambda \sum_{j=1}^{d} \beta_j^2$	Smooth shrinkage of slopes
Lasso penalty	$\lambda \sum_{j=1}^{d} \lvert\beta_j\rvert$	Can set slopes exactly to zero

The exponential expressions above state the mathematics; use the stable softplus and log-sum-exp evaluations described earlier when computing them. Choose the representation and regularization strength using validation data, then evaluate the selected procedure on the final test set.

Quick Review and Terminology

Table 8. Core concepts in machine learning, data representation, and linear algebra.

Term	Meaning
Feature	Input property describing an observation
Target / label	Output variable to be predicted; a label is an observed target entry
One-hot encoding	k-dimensional binary representation with exactly one active coordinate
i.i.d.	Independent and identically distributed sampling assumption
Spanning set	Vectors whose linear combinations generate a space/subspace
Basis	Linearly independent spanning set; minimal representation of a vector space
Orthonormal basis	Basis vectors are mutually orthogonal and have unit norm
Encoding	Coordinate vector $\boldsymbol{w}$ representing a point relative to chosen spanning vectors
Decoding	Reconstruction $C\boldsymbol{w}$ in the original data space
Projection	Closest representable point in a lower-dimensional subspace, illustrated by $CC^\top \boldsymbol{x}$ for an orthonormal subspace basis
Supervised learning	Learning a mapping from labeled input/output examples
Unsupervised learning	Learning structure from data without target labels
Reinforcement learning	Learning a policy through state/action/reward interaction
Gradient	Vector of partial derivatives; at nonzero gradient, its direction gives steepest local ascent under the Euclidean norm
Automatic differentiation	Derivative evaluation by applying differentiation rules through a computation graph (exact up to floating-point arithmetic)
Reverse mode / backpropagation	Backward accumulation of sensitivities from a scalar output toward many inputs
Gradient descent	Iterative update $\boldsymbol{w}^{(k)} = \boldsymbol{w}^{(k-1)} - \alpha \nabla g(\boldsymbol{w}^{(k-1)})$
Covariance matrix	Empirical matrix $\Sigma = \frac{1}{n} X^\top X$ describing variances and covariances of mean-centered data under the $1/n$ normalization
Principal-component direction	Covariance eigenvector defining a direction of maximal remaining variance
Principal-component score	Coordinate obtained by projecting a sample onto a principal direction, e.g. $z_j = \boldsymbol{v}_j^\top \boldsymbol{x}$
PCA	Unsupervised linear dimensionality reduction using the leading covariance eigenvectors

Table 9. Core concepts in linear prediction, model selection, and evaluation.

Term	Meaning
Hypothesis space	Family of candidate prediction functions
Population / empirical risk	Expected loss under the data distribution / average observed loss
Design matrix	Matrix with one observation per row and one input feature per column, including an intercept column when used
OLS	Least-squares fitting of a model linear in its coefficients
Feature map	Transformation of inputs before fitting, for example polynomial expansion
Likelihood / NLL	Probability or density of observed targets as a function of parameters / its negative logarithm
Logistic regression	Bernoulli model with a linear log-odds score
Softmax	Normalized exponential scores defining multiclass probabilities
Cross-entropy	Classification NLL written using binary or one-hot target indicators
Bias / variance	Systematic error of the average fit / variation in predictions across training sets
Ridge / lasso	Squared-coefficient / absolute-coefficient regularization
Hyperparameter	Setting selected outside coefficient fitting, such as regularization strength
Cross-validation	Repeated fitting and validation on complementary data folds
Precision / recall	Fraction of predicted positives that are correct / fraction of actual positives found
ROC AUC	Ranking performance summarized across binary classification thresholds

References

[1] Deisenroth, Faisal, and Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020.

[2] Simon Prince. *Understanding Deep Learning*. MIT Press, 2023.

[3] Watt, Borhani, and Katsaggelos. *Machine Learning Refined*, Chapters 2, 3, and 8, and Appendix B.

[4] Caron et al. (2018). *DeepCluster: Deep Clustering for Unsupervised Learning of Visual Features*.

[5] 3Blue1Brown. *Essence of Linear Algebra*, video series.

[6] Indro Spinelli. *Linear Models*. Foundations of Data Science lecture slides, Sapienza University of Rome, 2026. Source for Part V (slides 2–88).

[7] Goodfellow, Bengio, and Courville. *Deep Learning*. MIT Press, 2016, Chapter 5.

[8] Hastie, Tibshirani, and Friedman. *The Elements of Statistical Learning*. Second edition, Springer, 2009, Chapters 3–4.

[9] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006, Chapters 3–4.